Evidence, not verdicts: what 590 public PX4 crash logs can and cannot tell a drone fleet operator

Nagesh Chougule[a,*], Vibhav Ambardekar[a], Aditya Pathare[a], Atharv Waghmare[a], Shreyash Pawar[a], Piyush Bhamare[a]

[a]*Department of Mechanical Engineering, COEP Technological University, Pune, India*

Abstract

Flight logs are expected to show whether the pilot or the aircraft caused a drone crash, and which drones will fail next. We tested both on 590 crash logs from a 479,547-flight PX4 archive, declaring confirmatory tests in advance (†marks later exploratory analyses). A fixed attribution rule reached 0.47 balanced accuracy on 112 held-out crashes (pass mark 0.70); †models stayed near chance (AUC 0.42–0.59), though 36 pilot-labelled crashes could detect only AUC ≥ 0.72. †Across 32 evaluation settings, apparent crash-detection AUC fell by 0.21 [0.06, 0.34], 0.06 of it from grouping folds by vehicle; crash-window leakage was small overall but material among multicopter crashes with a crash-like tilt (0.089 [0.019, 0.168]; post hoc). †Two external checks bound these results. On 488 real flights with injected faults, our features separated faulty from fault-free flights at AUC 0.86 before any fault was injected, and the rule called only 3% of motor faults aircraft-caused; a published log checker separated crashes from other drones' flights but not causes. †Archive records are weak: 61% of crash-rated multicopter logs show no crash-like tilt; pilot-rated logs show one less often (adjusted odds ratio 2.1 [1.2, 3.7]); and a blind non-author reviewer agreed with uploader ratings no better than chance ($\kappa = -0.10$). Crash flights showed no warning against the drone's own earlier flights. Drones that later crashed vibrated more than matched drones ($^{\dagger}p = 0.012$ per airframe, Holm 0.08), but against 412 non-crash drones vibration ranking did not clearly beat random inspection. We give a reporting checklist and an evidence-first assurance design.

Keywords: Unmanned aircraft systems, Flight logs, Crash investigation, Fault diagnosis, Data leakage, Fleet assurance

*Corresponding author.
Email address: nkc.mech@coeptech.ac.in (Nagesh Chougule)

1. Introduction

Small unmanned aircraft are now flown commercially for survey, inspection, agriculture and delivery, and the rules that govern them place weight on records. In the European Union and the United Kingdom an operator in the specific category must keep records of operations, maintenance and personnel for at least three years [1, 2]; India requires an accident to be reported within 48 hours [3], and the United States within ten days [4]. When something goes wrong, the onboard flight log is the richest record available. Open-source log viewers, commercial fleet platforms and, more recently, machine-learning and language-model prototypes promise to turn that log into an answer: what failed, and whether the pilot or the aircraft was responsible [5, 6].

The question matters beyond engineering curiosity. In aviation safety science, accidents are recognized as systemic, multi-causal events involving human factors, environmental conditions and organizational influences [7]. Commercial warranty claims, insurance settlements and regulatory filings nevertheless frequently compress these dynamics into a binary decision: whether the pilot or the aircraft was primarily responsible. An automatic verdict that looks objective can carry weight it has not earned. Fleet operators face a related but different question: not why one flight ended badly, but which of their drones are most likely to fail next and should be inspected first. A log-based method that is good at the second task need not be good at the first, and confusing the two can mislead both investigators and maintainers.

Whether logs can deliver on either expectation has, to our knowledge, not been tested at scale against real crashes with evaluation protocols that guard against the two known sources of optimism. First, a log that contains a crash also contains its consequences—saturated motors, extreme attitudes and estimator errors—so a detector can learn the crash itself rather than its cause, a form of crash-window data leakage [8, 9]. Second, comparing crash flights with other operators' good flights mixes the flight with the aircraft and the people using it, so a difference between groups need not say anything about the flight that crashed (between-aircraft confounding). Public fault datasets are dominated by simulation or injected faults [10, 11, 12], and accident statistics classify outcomes such as loss of control rather than causes visible in a log [13, 14]; neither resource tests whether real crash logs, evaluated under leakage-free and within-aircraft protocols, can support the attribution and prioritization decisions that operators actually need.

The central question this paper asks of the largest public archive of real

autopilot logs, PX4 Flight Review [15, 16], is: *what evidentiary value can flight logs carry for a drone fleet operator, once evaluation protocols guard against crash-window leakage and between-aircraft confounding?* Both were tested. Crash-window leakage was small for model accuracy overall (0.014 of AUC in the factorial audit), possibly in part because most crash logs have no clear crash moment; on multicopter crashes with a crash-like tilt it was material (0.089 [0.019, 0.168]) and did not depend on the tilt features themselves; grouping cross-validation folds by vehicle mattered more, and comparing each drone with its own earlier flights mattered most. A further threat to validity, which we had not planned to test, lies in the archive's own records: most crash-rated logs show no crash-like event, the uploader's label is associated with whether one is present, and dates cannot order flights within a day. These defects can bias results in either direction, and date ordering matters only to longitudinal designs. We decompose this into four sub-tests:

- Q1 Do configuration settings differ between crash flights and normal flights, and can that difference indicate who was responsible?
- Q2 Can a log separate crashes caused by the pilot from crashes caused by the aircraft—and does this separation survive expert-diagnosis validation?
- Q3 Once within-aircraft and between-aircraft comparisons are distinguished, which comparison retains a usable signal of pre-crash deterioration?
- Q4 Do expert diagnoses agree with uploader labels, and do automatic diagnostic checks find the faults experts name without benefiting from the crash window?

Because language-model assistants are now proposed for exactly this task, we also report a controlled evaluation of small local models used as an attribution review panel, including a test for voting-consensus manipulation.

The contributions fall into two groups. The first concerns how log-based crash diagnostics should be evaluated on public archives: (i) †*Evaluation choices.* A factorial audit of 32 evaluation settings: apparent crash-vs-normal AUC fell by 0.21 [0.06, 0.34] (0.88 to 0.67), of which 0.06 came from grouping folds by vehicle, the only factor that could be isolated (Section 5.5). (ii) †*Crash moments.* In 61% of crash-rated multicopter logs there is no crash-like tilt; three raters (two authors, one of them blind, and a non-author) agree that the computed crash moment is mostly right when tilt-defined and mostly wrong otherwise (Fleiss' $\kappa = 0.71$), so crash-window analyses rest on

a minority of logs (Section 5.4). (iii) †*Label confound.* Pilot-rated logs show a crash-like tilt half as often as aircraft-rated ones (23% against 45%; odds ratio 2.1 [1.2, 3.7] adjusted for era and vehicle type), so the label partly encodes whether the log shows a tilt-type crash and a classifier could separate the labels without learning anything about cause; in our modelled multicopters tilt alone reaches only AUC 0.53, and attribution stayed near chance within each tilt stratum (Section 5.4). (iv) †*An independent tool.* A published open-source log checker, run unchanged under the same protocol, separated crash flights from other drones' flights (AUC 0.72 without the last 10 s) but gave no within-drone warning and attributed no better than chance (balanced accuracy 0.52), so the pattern is not an artefact of our features (Section 5.8). (v) †*A known-cause control.* On 488 real flights with firmware-injected faults of known onset, our features separated faulty from fault-free flights at AUC 0.86 using only data from before the fault, and the declared rule called 3% of motor faults aircraft-caused: a high fault-versus-normal AUC can arise with no fault present, and the rule's signs miss faults the controller absorbs (Section 5.9). (vi) †*Flight order.* Dates cannot order flights within a day: by GPS time, 44–55% of dated crash-day "earlier" flights were flown after the crash; removing them left the declared conclusions in place but removed an apparent within-drone effect, which we withdraw (Section 5.6).

The second concerns what logs can tell a fleet operator: (vii) With these features, rules and labels, logs did not separate pilot-caused from aircraft-caused crashes: the declared rule reached 0.47 balanced accuracy on held-out crashes, and exploratory models gave AUC 0.42–0.59 with approximate upper bounds ≤ 0.69, below the ≈ 0.8 a verdict needs. †A blind reviewer who is not an author agreed with the uploader ratings no better than chance ($\kappa = -0.10$), and the rule did no better against the reviewer's labels (0.55), so the failure is not an artefact of noisy uploader labels (Section 5.2). (viii) Drones that later crashed vibrated more than matched drones on ordinary flights (mean rank 0.67; $p = 0.012$ clustered by airframe, Holm-adjusted 0.08), but against 412 non-crash reference drones a vibration ranking was not shown to beat random inspection (lift 1.77× [1.00, 2.60] at a measured 13.7% flag rate), so it is not a crash warning and at most a weak inspection aid (Section 5.6). (ix) A small validation set of real PX4 crash logs with independent expert diagnoses (148 threads coded twice, $\kappa \geq 0.73$; 43 downloadable logs, 9 with uploader labels), a reporting checklist for log-based diagnostics (Section 6.1), and an evidence-first design for fleet-assurance tools. Code, declarations, derived features and labels are released.

2. Related work

Fault datasets. Table 1 compares our data with the main public resources. ALFA provides 47 fixed-wing flights with injected engine and control-surface failures [10]. RflyMAD covers 5,629 multicopter fault cases with eleven fault types, of which 497 are real flights and the rest software- or hardware-in-the-loop simulations [11]. UAV-SEAD offers about 1,396 real PX4 logs annotated for state-estimation anomalies [12], used by later anomaly-mining work [17]; a recent testbed quantifies the discrepancies between simulated and real PX4 logs [18]. These resources target anomaly or fault detection under known faults. Wang et al. collected 178 real abnormal PX4 and ArduPilot logs from software-bug incidents and studied those that led to crashes [19]; their labels are software root causes. To our knowledge, none of these resources labels real crashes by pilot-versus-aircraft responsibility or separates the pre-crash window from the crash.

Fault detection and diagnosis. Surveys of fault detection for UAVs and robots cover model-based and data-driven methods and stress how hard robotic diagnosis is in the field [20, 21]. Causal models built from autopilot documentation have been used to find actual causes of failures in ArduPilot logs [22]. A study of 569 bugs in PX4 and ArduPilot found parameter handling among the root-cause types of drone-specific bugs [23], and a later study collected 178 abnormal PX4 and ArduPilot flight logs from real software-bug incidents [19]. MAYDAY argues that flight logs record only high-level control states, so an accident whose cause lies in the control program often cannot be traced from the log alone; it adds program-level logging and traced 10 ArduPilot accidents to control-semantic bugs [24]. Our negative attribution result is the statistical counterpart of that argument for pilot-versus-aircraft calls. Recent work recovers and parses damaged ArduPilot, PX4 and Betaflight logs for forensic use [25], and diagnoses battery-induced crashes from hexacopter flight logs in controlled scenarios [26]; both study logs under known conditions rather than an uncurated archive. Gryphon extracts critical events from ArduPilot logs with a six-step forensic method that ends in a timeline [27]; AeroFleet's evidence timeline follows the same principle and adds no attribution. Maintenance-oriented reliability analysis of UAVs models the passage from partial degradation to failure [28], the view our between-drone result supports.

Table 1: Public UAV fault and anomaly data compared with this study.

Resource	Flights	Faults or labels	Real crashes labelled by cause (to our knowledge)
ALFA [10]	47 fixed-wing flights	Injected engine and control-surface failures	No
RflyMAD [11]	5,629 cases (497 real)	11 injected fault types	No
UAV-SEAD [12]	about 1,396 real PX4 logs	State-estimation anomalies (4 classes)	No
Wang et al. [19]	178 real abnormal PX4/ArduPilot logs	Software-bug root causes	Partly (software bugs only; not responsibility)
This study	479,547-log index; 590 crash-rated logs; 148 expert-diagnosed threads	Uploader responsibility labels; expert causes	Yes (uploader; partly expert-checked; to our knowledge)

Accident analysis. Analyses of civil drone occurrences find equipment problems and loss of control prominent [13]. Human-factors frameworks for remotely piloted aircraft (such as the Human Factors Analysis and Classification System, HFACS) model mishaps as multi-causal sequences involving organizational influences, environmental preconditions, and supervisory failures alongside direct operator actions [7, 29]. In operational practice, post-incident reporting and warranty claims often collapse these complex systemic interactions into a single nominal party: pilot versus aircraft. Because such classifications depend heavily on the rater and framing, we evaluate agreement against expert community diagnoses.

Language models for diagnosis. Language-model agents have been applied to simulation testing and log analytics for small UAVs [5] and to UAV forensic investigation [30], and learned models to classifying problems in drone log messages for forensics [6]. Multi-agent debate has been proposed to improve the factuality of model outputs [31], while indirect prompt injection shows that text inside data can steer model behaviour [32]. We test both properties

on our task.

Methodology. Leakage, where information unavailable at prediction time enters training or evaluation, is a known source of irreproducible machine-learning results [8, 9]; in bearing fault diagnosis, splitting by segment rather than by physical bearing inflates reported accuracy, and bearing-wise partitioning is proposed as the fix [33]. Grouping folds by unit is therefore not new; what we add is a measurement of how much it, and four other evaluation choices, change results on real crash logs; label noise likewise biases evaluation [34]. In the drone-log domain we identify a domain-specific variant: *crash-window leakage*, in which the physical consequences of the crash (saturated actuators, estimator divergence, attitude extremes) contaminate features used to evaluate pre-crash diagnostics. Matching reduces confounding in observational comparisons [35], and declaring hypotheses and analysis scripts in version control prior to running tests separates confirmatory from exploratory analysis [36]. We apply these ideas together as an evaluation protocol for drone logs.

Literature search. One author searched IEEE Xplore, the ACM Digital Library, ScienceDirect, arXiv and Google Scholar for 2016–2026 with three query strings (crash logs and attribution; leakage and cross-validation in UAV anomaly detection; vibration-based maintenance), recorded with the inclusion rules in the released literature-search protocol (26 October 2026). Four closely related studies [24, 27, 19, 33] were missed by that search and added on 27 October. Forward citation chasing on three of them (Semantic Scholar, 27 October 2026) added three more [26, 25, 30]; chasing for MAYDAY could not be completed through the public API. The search is not a systematic review.

3. Data

3.1. Archive and labels

PX4 Flight Review is a public service to which PX4 users upload onboard logs (ULog format) for plotting and sharing [16]. We downloaded its public index on 20 October 2026: 479,547 logs (32,793,172 bytes, SHA-256 `1bbb01c3…a92d6`), of which 442,929 came from real hardware rather than simulation, from 38,320 distinct vehicle identifiers. Uploaders may rate a log. A total of 590 logs carry an uploader crash rating (402 aircraft-caused,

188 pilot-caused); 70 of them come from simulation, and Table 2 details the 520 from real hardware. These uploader-assigned responsibility labels are not independently verified ground truth; they reflect the uploaders' own assessments. A notable discrepancy appears between uploader ratings and expert conclusions: while uploaders attributed 31.9% of rated crashes to the pilot (188 of 590), forum experts blamed the pilot in only 6.1% of all threads (9 of 148) and 8.6% of decided threads (9 of 105, Section 5.10). Across all 148 real crash threads, download attempts yielded 43 downloadable logs from 43 distinct threads (38 aircraft-fault and 5 pilot-fault; the remaining 105 had unhosted or expired external links). Cross-tabulating these 43 logs against archive ratings reveals that 9 carried uploader aircraft crash ratings (`crash_sw_hw`, with 100% expert agreement: all 9 diagnosed as aircraft faults), 33 were unrated in the archive index (28 aircraft, 5 pilot), 1 was rated unsatisfactory (aircraft), and exactly 0 were uploaded with a `crash_pilot` rating. This pattern is consistent with, but not statistical evidence of, reluctance to self-identify pilot fault: all 5 pilot logs were unrated in the index against 28 of 38 aircraft logs unrated (one-sided Fisher exact $p = 0.25$). Across the wider set of all 148 forum threads, 14 referenced logs that carried an uploader aircraft crash rating in the archive index (experts agreed with the aircraft fault in 11 and were unclear in 3; none blamed the pilot); 9 of these 14 threads were among the 43 with surviving, downloadable log files (all 9 with complete expert agreement), while 5 had expired external links. This gap may reflect selection bias in which incidents operators submit for community assistance, or a narrower definition of pilot error among autopilot developers compared to end users. Furthermore, forum diagnoses represent informed community opinion rather than independently verified accident inspection ground truth; original posters explicitly confirmed the diagnosis in only 47 of 94 aircraft threads and 5 of 9 pilot threads. Logs rated "good" or "great" served as normal flights. Logs were parsed with pyulog. We store derived features and log identifiers, not the logs.

A ULog file contains the parameters in force when logging started, any parameter changes during logging, timestamped messages published by the autopilot's modules, and logged text messages with a severity level. The analyses below use the vehicle status (arming state, navigation mode, failsafe flags), attitude and body-rate topics, rate setpoints, inertial-sensor status, actuator outputs, battery status, GPS position and the land detector. The vehicle identifier in the index is derived from the flight controller's hardware serial number; it identifies a flight controller, which in practice usually means

Table 2: Composition of crash-rated and well-rated logs from real hardware in the archive.

	Crash-rated	Rated good or great
Logs	520	21,112
Median duration	54 s	388 s
Most frequent years	2018 (91), 2025 (89), 2019 (87), 2017 (57)	2018 (10,571), 2017 (5,748), 2019 (2,910)
Most frequent PX4 versions	1.8 (70), 1.9 (66), 1.15 (65), 1.11 (65)	1.7 (6,299), 1.8 (5,785), 1.6 (4,265)
Vehicle types	quadrotor 320, fixed wing 71, hexarotor 38	quadrotor 19,890, fixed wing 620, hexarotor 291

one airframe, though a controller can be moved between frames.

Crash-rated logs differ from well-rated logs in composition (Table 2): they are shorter (median 54 s against 388 s, partly because a crash ends the log) and spread over more firmware generations. All between-group comparisons below therefore use either matching or within-drone designs, and we do not pool across firmware generations where measurements differ.

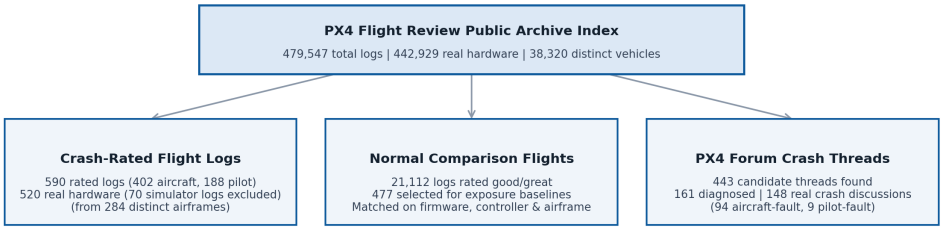
3.2. Samples

Figure 1 and Table 3 show how each analysis sample was drawn. We evaluate two primary classification tasks: Task A (separating crash flights from normal flights) and Task B (binary attribution of crash responsibility between drone and pilot). We distinguish two distinct cohorts that each happen to number 148: (1) the leak-curve and attribution modeling sample of 148 crashes and 139 normal flights at $m = 0$ s (Section 4), and (2) the expert community sample of 148 crash discussion threads on the PX4 forum (Section 5.10). The attribution rule was evaluated on 205 crashes (93 development and 112 held-out). All 148 crashes entering the statistical attribution models are a strict complete-case subset of these 205 crashes (100% overlap); 57 crashes were excluded from modeling because required ULog topics were unrecorded or the flight window before cut-off was shorter than 10 s. In the leak-curve and attribution modeling sample, sample sizes across cut-off margins $m \in \{0, 2, 5, 10, 20, 30\}$ s reflect progressive attrition as flights with duration under 10 s before cut-off are dropped: for Task A (crash vs. normal), 148 vs. 139 ($m = 0$ s), 142 vs. 137 ($m = 2$ s), 135 vs. 133 ($m = 5$ s), 114 vs. 130 ($m = 10$ s), 100 vs. 128 ($m = 20$ s), and 89 vs. 124 ($m = 30$ s); for Task B (drone vs. pilot responsibility among crashes), 112 vs. 36 ($m = 0$ s,

$n = 148$), 106 vs. 36 ($m = 2\,\text{s}$, $n = 142$), 101 vs. 34 ($m = 5\,\text{s}$, $n = 135$), 87 vs. 27 ($m = 10\,\text{s}$, $n = 114$), 73 vs. 27 ($m = 20\,\text{s}$, $n = 100$), and 63 vs. 26 ($m = 30\,\text{s}$, $n = 89$). A constant complete-case subset of 213 flights (89 crashes and 124 normal flights; 63 drone and 26 pilot crashes) with pre-crash duration $\geq 40\,\text{s}$ was evaluated across all six margins without attrition.

Initial cross-sectional samples for Q1 and Q2 selected flights with PX4 1.10 or newer and at least 5 s of flight; normal flights were matched to crashes on vehicle type and drawn from vehicles without any crash rating. For the longitudinal within-drone and between-drone tests (Q3b, Q3c), only the eligible case count ($n = 12$), not any outcome or feature data, was inspected before deciding to lift the firmware restriction to avoid sample starvation; flights were instead strictly matched within each drone's firmware generation and measurement source. For the within-drone test every crash with a vehicle identifier and at least three earlier non-crash flights of 30 s or more was eligible (101 crashes; up to the five most recent earlier flights each). For the between-drone test each crashed drone was matched with up to two drones without any crash-rated log that shared the flight-controller hardware, the PX4 major.minor version and the vehicle type (66 matched sets, 124 control slots drawn from 107 unique non-crash control drones, three randomly chosen flights each, seed 7). Control drones were sampled without replacement within each matched set. The three exposure flights of a crashed drone were its most recent non-crash flights dated on or before the crash; flights on the crash day were included in the declared test because the archive does not give their order within a day, and removed in a sensitivity analysis.

Not all selected flights enter every test. In the within-drone candidate pool (101 crashes with earlier flights), 34 crashed drones were dropped because they lacked multiple valid flights or had duration under 10 s before cut-off, leaving 67 multi-flight drones (372 total flights: 67 crash and 305 earlier flights). Among these 372 flights, 55 individual flight logs could not be read or had under 10 s before cut-off; requiring that crash and earlier flights share the same measurement source left 61 analysed drones for vibration, and restricting to rotary-wing vehicles with rate setpoints left 46 analysed drones for rate tracking. In the between-drone sample (66 matched sets), 71 flights were unreadable or too short; a drone's value requires at least two readable flights, and case and controls must share the measurement source, leaving 60 analysed sets for vibration and 47 for tracking. In the between-drone sample, 124 matched control slots contributed 372 flights (124×3), distinct from the 372 flights (67 crash flights and 305 earlier flights)



Analysis Stream	Initial Candidate Pool	Exclusions & Attrition	Analyzed Sample Size
Q1: Parameters	100 crash, 477 normal	43% crash logs lack factory defaults	57 crash, 113 normal
Q2: Fixed Rule	205 crashes, 143 normal	None (evaluates 93 dev + 112 held-out)	205 crashes, 143 normal
Q2/Q3a: Leak Curve	205 crashes, 143 normal	57 dropped (missing topics / window < 10 s)	148 crash, 139 normal (m=0 s) 89 crash, 124 normal (m=30 s)
Q3b: Within-Drone	101 crashed drones	34 dropped (single flight / window < 10 s)	67 multi-flight drones (372 fl.) 61 vibration, 46 tracking
Q3c: Between-Drone	66 matched sets (101 cases)	6 sets dropped (unmatched sensor metric)	60 vibration sets (124 ctrl, 372 fl.) 47 rate-tracking sets
Q4: Expert Cases	148 real crash threads	105 logs unhosted or expired links	43 downloadable (38 aircraft, 5 pilot) 35 aircraft with fault categories

Figure 1: From the public index to each analysis. Counts are logs unless stated.

evaluated in the within-drone longitudinal model.

3.3. Expert diagnoses

We searched the PX4 discussion forum for threads that link a Flight Review log and mention a crash (443 threads, 410 with a log link). For the 161 threads with a reply from PX4 staff, the development team or a member at the forum's highest trust levels (148 of them describing a real crash), we recorded the expert's conclusion (aircraft, pilot, both, unclear, no diagnosis), the cause named, who gave it, whether the original poster confirmed it, and the sentence stating the diagnosis. Recording was AI-assisted and reviewed by an author; 123 of 124 recorded sentences were found verbatim in their thread by script. A second author then coded all threads blind to the first recording, with the same categories; a script confirmed that 150 of 151 coded rows quoted their own thread or recorded no diagnosis. On the 147 threads both coded, agreement (Cohen's kappa, 95% bootstrap interval) was 0.81 (0.71–0.88) for the expert's conclusion, 0.73 (0.65–0.81) for the cause and 0.79 (0.69–0.88) for whether the poster confirmed it, above the declared pass mark

Table 3: Analysis samples and evaluation cohorts.

Sample	Crashes	Comparison	Question
Settings	100 (57 with default records)	477 normal (113 with default records)	Q1
Development crashes	93 analysed	–	Q2 rule
Held-out crashes	112 analysed	143 normal (full logs)	Q2 replication
Leak-curve set	148 at $m = 0$ s (89 at $m = 30$ s)	139 normal (124 at $m = 30$ s)	Q2 model, Q3a
Constant-subset leak curve	89 crashes (≥ 40 s duration)	124 normal ($n = 213$ constant across margins)	Q2 model, Q3a
Within-drone rank test	101 selected (61 and 46 analysed)	475 earlier flights of the same drones	Q3b
Within-drone predictive model	67 multi-flight drones (67 pre-crash)	305 earlier flights of same drones ($n = 372$)	Q3b
Between-drone matched sets	66 matched sets (60 and 47 analysed)	124 matched drones, 372 flights	Q3c
Expert forum cases	148 real crash threads	43 logs downloadable (38 aircraft, 5 pilot)	Q4

of 0.60. The two coders settled the 45 threads that differed on conclusion or cause by discussion, which changed 13 labels; counts and tests below use the settled labels, and the first recording led to the same conclusions (both are in the repository). The searches combined the Flight Review host names with the words crash, crashed, flyaway, fell, flipped and lost control, and every returned thread was retrieved through the forum's public interface. A

reply counted as expert when its author was marked as staff, belonged to the development team group or held one of the forum's two highest trust levels. Only public posts were used; usernames and post text are not redistributed.

4. Methods

4.1. Declared test protocols

The public archive index was downloaded on 20 October 2026. For each confirmatory test, the hypothesis, sample selection, features, statistic, and pass mark were declared and committed to version control before the analysis it governs was run (Table 4). Where a test reused logs already downloaded for an earlier test, the protocol records it as declared before running, not before download (exploratory analyses added after the declared protocol are marked with †). Unlike third-party timestamped registries (such as the Open Science Framework), commit timestamps in self-hosted git remain author-controlled until an independent snapshot is indexed; the git commit history establishes an immutable cryptographic SHA-1 chain in version control, and archived snapshots have been deposited in Software Heritage (`swh:1:rev:bad06ff1256865c7e63d44389aac174e0281291d`, `swh:1:rev:bbdd0420de4547ac7d67b84165292a46237b604d`, `swh:1:rev:624dd694e0d207dd9c44b1c9550faaf04453bc56`, and `swh:1:rev:3a9aa89417fa9ea2a2334e3a4794e2358fb713ee`) alongside the repository's git commit hashes (including `eb463b5` for the decomposition), and will be minted into a Zenodo archive DOI upon publication. One amendment was data-dependent: after observing that the firmware filter of the within-drone test left only 12 eligible cases under the initial ≥ 1.10 filter (without inspecting any outcome data), the firmware filter was removed and a fallback for older firmware was added. One coding error was corrected after a first run (logs whose CPU-load record lacked the load field were being skipped); no threshold changed. Secondary sensitivity analyses and diagnostic evaluations added after the declared protocol are explicitly designated as exploratory and documented with commit hashes in Appendix A.

4.2. Crash moment and analysis window

The crash moment t_c is the first time after take-off at which the tilt $\theta = \arccos\big(1 - 2(q_x^2 + q_y^2)\big)$, computed from the attitude quaternion, exceeds $60°$; when no such tilt occurs, it is the end of the armed period. The tilt rule is skipped only when the log's vehicle-status record marks the aircraft as

Table 4: Declaration and results commits for each test (IST, from the git history; author-controlled until archived). Each test was run between its declaration commit and the commit that first added its result file, so the run time is bounded by those two times. Tests sharing a script share its declaration (for example, the clipping and manual-mode sign tests are in the Q2 rule script; within- and between-drone tracking are in the Q3b and Q3c scripts).

Test	Focus (script)	Declared	Results committed	Status
Q1	Protections disabled, Fisher exact (`analyze.py`)	`d755cca` 20 Oct 17:23:31 (code and result together)		First pass, not declared before running
Q2 dev	Attribution rule, clipping and manual-mode signs (`pilot_vs_drone.py`)	`63bc72e` 20 Oct 17:25:49	`19a9318` 17:26:57	Confirmatory
Q2 held	Held-out rule replication (`holdout_test.py`)	`af6fab2` 20 Oct 17:28:42	`da92ec3` 17:40:02	Confirmatory
Q4, $m = 0$	Expert fault checks (`test_flight_check.py`)	`4475c8c` 20 Oct 21:39:26	`da5038b` 21:46:47	Confirmatory
Q4, $m = 10$ s	Same checks, last 10 s removed	`0c8ee47` 20 Oct 21:46:54	`95d873a` 21:48:02	Confirmatory
Q3a	Leak curve, $\Delta \geq 0.10$ (`leak_curve.py`)	`d5d56c7` 23 Oct 11:17:11	`1ecb4c9` 11:19:07	Confirmatory
Q3b	Within-drone vibration and tracking (`within_drone.py`)	`8b417d2` 23 Oct 11:21:19; amended `f911bcf` 11:22:21	`c9ebca4` 12:19:44	Confirmatory (amended before download)
Q3c	Between-drone vibration and tracking (`rough_drone_test.py`)	`d7a6574` 23 Oct 12:30:19	`a76babb` 18:15:54	Confirmatory
Replication	Newer-metric vibration (`rough_drone_newer.py`)	`2148644` 24 Oct 00:00:48	`b25d596` 00:40:24	Confirmatory
Coding	Second coder, $\kappa \geq 0.60$ (`score_forum.py`)	`21af373` 24 Oct 00:03:29	`a369f83` 12:16:14	Confirmatory

not rotary-wing. Older logs lack that flag and are treated as rotary-wing, so the rule was also applied to some fixed-wing and VTOL logs, where a bank above 60° can be normal flight. Aircraft type is taken from the archive index: multicopter means quadrotor, hexarotor or octorotor; fixed-wing means fixed wing; VTOL means any index type marked as vertical take-off and landing (tailsitters, tiltrotors and fixed-wing aircraft with lift rotors), which hover like multicopters but fly forward on a wing. Crash-moment analyses are therefore reported for multicopters (Section 5.4). Normal flights use the end of the armed period. Take-off is the first sample after arming at which the land detector reports the vehicle airborne. An analysis with cut-off m uses the window from take-off to $t_c - m$ and is skipped if the window is shorter than 10 s. Because normal flights include the complete descent, landing and disarm sequence while crash windows terminate at impact or tilt, this introduces an

Table 5: Features computed on each analysis window.

Feature	Definition
Rate-tracking error	RMS of commanded minus measured body rate, roll and pitch ($\mathrm{rad\,s^{-1}}$; rotary wing)
Tilt (p95)	95th percentile of tilt angle (deg)
Vibration (p95)	95th percentile of the accelerometer vibration metric, or of the estimator vibration estimate on older firmware
Clipping rate	accelerometer clipping events per second
Motor saturation	share of time with any motor output at 95% or more (rotary wing)
Battery minimum	lowest reported remaining charge (%)
Voltage sag	(start voltage − minimum voltage) / start voltage
Poor GPS	share of time with horizontal accuracy above 5 m or fewer than 6 satellites
Failsafe	1 if a failsafe was active at any time
Manual share	share of time in a pilot-flown mode (manual, altitude, position, acro, stabilised)
Stick deflection (p95)	95th percentile of the larger roll or pitch stick deflection

inherent windowing asymmetry that contributes to differences in duration and whole-window aggregates.

4.3. Features

Table 5 defines the features. Vibration is not measured the same way across firmware generations: newer logs report an accelerometer vibration metric, older ones an estimator vibration estimate. Comparisons are therefore only made between flights measured with the same source, and the source is recorded with every value. The rate-tracking error falls back to the attitude message's body rates when the angular-velocity topic is absent.

4.4. Tests

Q1 settings. For logs that carry the firmware's record of default values, a protection was counted as disabled when its value differed from the default and switched it off (for example a failsafe action of "none"). Crash and normal logs were compared with Fisher's exact test.

Q2 attribution. Task B is defined as the binary attribution of crashes between pilot and aircraft fault (contrasted with Task A, which evaluates separating crash flights from normal flights). The declared rule labelled a crash "aircraft" if any of eight signs fired before t_c (a critical battery warning or remaining charge below 10%; any motor at 95% or more for at least 1 s in the last 10 s; more than 50 clipping events; poor GPS in a GPS-dependent mode; a failure-detector flag; an error-level message; CPU load above 95%; an active failsafe), otherwise "pilot" if the mode at t_c was pilot-flown, otherwise "unknown". The pass mark was a balanced accuracy of at least 0.70 on decided cases, answering at least 60% of crashes and being wrong on at most 20%. A model-based check used L2-regularised logistic regression on all features ($\lambda = 1$, standardised inputs, median imputation within training folds), evaluated by stratified five-fold cross-validation repeated five times and summarised by the ROC AUC [37]. As an exploratory check on non-linear threshold effects, single-split decision stumps were evaluated alongside logistic regression. We also evaluated depth-3 decision trees across all cut-offs ($m \in \{0, 2, 5, 10, 20, 30\}$ s). Logistic models, stumps and trees are exploratory; their uncertainty is assessed against a permutation null that refits the whole cross-validation (Section 5.2).

The signs were chosen before the analysis to cover common failure modes of PX4 multirotors: power, propulsion, vibration, navigation, software and failsafes. The pilot rule was deliberately simple: if no aircraft sign fired and the vehicle was in a mode in which the pilot controls attitude or position directly, the crash was attributed to the pilot. The rule was fixed before it was run on any crash log; only the list of message types available in the logs had been inspected.

Q3a crash window. The same model separated crash from normal flights at cut-offs $m \in \{0, 2, 5, 10, 20, 30\}$ s. We declared that the AUC would fall by at least 0.10 between $m = 0$ and $m = 10$ s if crash consequences drove it.

Q3b and Q3c rank tests. For a crashed drone i with value x_{i0} and comparison values $x_{i1}, \ldots, x_{in_i}$, the scaled rank is

$$r_i = \frac{1}{n_i} \sum_{j=1}^{n_i} \mathbf{1}\left[x_{ij} < x_{i0}\right], \tag{1}$$

which is 0 when the crashed drone has the lowest value and 1 when it has the highest; ties count against the hypothesis. A rank statistic was chosen

for three reasons: the two vibration sources have different units and scales, so only orderings are comparable across drones; the number of comparisons differs between drones, which the exact null handles directly; and the distributions are skewed, with a few very rough flights, which would dominate a mean difference. In the within-drone test (Q3b) x_{i0} is the crash flight, cut 10 s before t_c, and the comparisons are the same drone's earlier flights, each cut 10 s before the end of its armed period. In the between-drone test (Q3c) x_{i0} is the median over the crashed drone's three most recent pre-crash flights and the comparisons are the medians of the matched controls. Under no difference each r_i is uniform on $\{0, 1/n_i, \ldots, 1\}$ with mean 0.5, and $\bar{r}$ estimates the probability that the crashed drone exceeds a comparison, the quantity estimated by the Mann–Whitney statistic [38]. One-sided p-values come from 100,000 draws of this exact null; 95% confidence intervals from 10,000 bootstrap resamples of drones [39]. For the proportion of sets in which the crashed drone is the roughest, the expected null rate depends on the mix of set sizes: among the 60 analysed vibration sets, 35 had two matched controls (null probability 1/3) and 25 had one control (null probability 1/2), giving an exact Poisson-binomial expected null proportion of $(35 \times \frac{1}{3} + 25 \times \frac{1}{2})/60 = 40.28\%$ (null mean 24.167, SD 3.745). For duration adjustment in sensitivity analysis, log(vibration) was regressed on log(flight duration) within each measurement source across all exposure flights, and the rank test was repeated on the residuals. Each test had two primary features (vibration, rate-tracking error) at $\alpha = 0.025$.

Q4 expert diagnoses. Expert conclusions were compared with uploader labels. Fixed deterministic checks for the five physical failure modes named by experts were evaluated: (i) *Tuning*: combined roll and pitch rate-tracking RMS error > 0.30 rad/s over at least 50 samples; (ii) *Vibration*: total accelerometer clipping counts > 50 or 95th-percentile `accel_vibration_metric` $> 30\,\mathrm{m/s^2}$; (iii) *GPS/Compass*: `mag_test_ratio` > 1.0 in `estimator_status` for ≥ 1.0 s cumulative, or horizontal position error eph > 5 m or satellites < 6 for ≥ 5.0 s cumulative while in a GPS-dependent mode; (iv) *Motor/ESC*: any motor output at $\geq 95\%$ authority (or PWM $\geq 1950\,\mu$s) for ≥ 1.0 s cumulative; (v) *Battery*: battery warning level ≥ 2 (critical/emergency) or state of charge $< 10\%$. Checks were declared useful if they fired on at least 50% of logs with that diagnosis and at most 10% of normal flights, evaluated first on the full window ($m = 0$ s) and then with the final 10 s removed ($m = 10$ s).

Language-model panel. The panel tested a design proposed for advisory diagnosis: several reviewers, each restricted to one slice of evidence (navigation, pilot inputs, GPS, battery, settings), vote on a fixed list of six causes and must cite numbered facts extracted from the log. Robustness was evaluated by replacing one reviewer with an adversary arguing for a chosen wrong cause, and planting hostile strings in uploader-controlled fields. Full details are provided in Section 5.11.

Multiplicity. Seven tests produce p-values. Holm's procedure [40] is reported over all seven in addition to each test's declared α. The Q1 settings comparison came from a first-pass analysis whose code and result were committed together (Table 4), so it is not confirmatory; it is kept in the Holm set, which makes the adjustment of the other tests conservative.

5. Results

Table 6 summarises the findings and how strongly each is supported; the subsections give the detail.

5.1. Configuration settings (Q1)

Of 57 crash logs and 113 normal logs carrying a firmware-default record, 29 (51%) and 52 (46%) had at least one protection disabled relative to the default (Fisher $p = 0.63$). This comparison comes from a first-pass analysis whose code and result were committed together, so it is reported as a descriptive result rather than a declared test (Table 4). Pilot-caused crashes had a protection disabled more often than aircraft-caused ones: 14 of 20 (70%) against 15 of 37 (41%), an observed difference of +29.5% (95% CI 3.9–55.0%, odds ratio 3.42, Fisher's exact two-sided $p = 0.052$). This contrast is exploratory, and does not reach statistical significance unadjusted (Fisher's exact two-sided $p = 0.052$). Furthermore, default parameter records were present only on newer firmware (missing for 43% of the 100 crash logs); on the 57 available cases, treating disabled protections as a pilot-attribution classifier yields a sensitivity of 70.0%, a specificity of 59.5%, and a balanced accuracy of 0.647, below the declared 0.70 pass mark. None of 100 crash logs changed a safety-critical or flight-limit setting during the flight.

Table 6: Main findings at a glance. "Declared": hypothesis and pass mark committed before the test ran; "exploratory": added after the declared protocol († in the text).

Question	Finding	Evidence	Verdict	Section
Pilot or aircraft? (Q2)	Rule 0.47 balanced accuracy on 112 held-out crashes; models AUC 0.42–0.59 (upper bounds ≤ 0.69)	Declared rule; exploratory models	No usable signal	5.2
Settings (Q1)	Protections disabled in 51% of crash and 46% of normal logs ($p = 0.63$)	First pass, descriptive	No difference	5.1
Expert agreement (Q4)	Rule vs. 43 expert-diagnosed logs: balanced accuracy 0.38 (CrI 0.25–0.61)	Exploratory	Inconclusive	5.10
Crash window (Q3a)	AUC drop 0.035 when the last 10 s are cut (criterion 0.10); window effect 0.089 [0.019, 0.168] on tilt-defined multicopter crashes ($n = 37$), 0.131 without tilt and stick features	Declared; post hoc subset	Small overall, material on tilt-defined multicopters	5.3, 5.4
Warning before a crash (Q3b)	Crash flight like the same drone's earlier flights (vibration rank 0.48, $p = 0.67$)	Declared	No warning	5.6
Which drones crash (Q3c)	Later-crashed drones vibrated more than matched drones (rank 0.67; clustered $p = 0.012$, Holm 0.08; one set per operator site $p = 0.14$)	Declared; clustered exploratory	Suggestive	5.6
Screening value†	Against 412 non-crash reference drones: nominal 10% threshold flagged 13.7%; lift 1.77× [1.00, 2.60]	Exploratory, declared	Not shown to beat random inspection	5.6
Evaluation choices	AUC 0.88 → 0.67 across 32 settings; 0.06 from grouping folds by vehicle	Exploratory	Fold grouping matters	5.5
Crash moment	No crash-like tilt in 61% of multicopter crash logs; fallback moment mostly wrong (three raters, Fleiss' $\kappa = 0.71$); two objective detectors failed validation	Exploratory	Unreliable for most logs	5.4
Label confound	Crash-like tilt in 23% of pilot-rated vs. 45% of aircraft-rated logs (adjusted OR 2.1 [1.2, 3.7])	Exploratory	Label tracks log content	5.4
Independent relabelling	Blind non-author reviewer vs. uploader: $\kappa = -0.10$ [−0.34, 0.17] on 58 crashes; rule vs. reviewer BA 0.55 [0.42, 0.66]	Exploratory, one reviewer	Labels contestable; failure not label-driven	5.2
Flight order	44–55% of dated crash-day "earlier" flights flown after the crash	Exploratory	Dates insufficient	5.6
Published log checker	FlightMD: crash vs normal AUC 0.72 [0.58, 0.81] without	Exploratory	Same pattern as our	5.8

5.2. Attributing a crash to the pilot or the aircraft (Q2)

The fixed rule reached a balanced accuracy of 0.57 on 93 development crashes (35% wrong) and 0.47 on 112 held-out crashes (47% wrong), against the declared 0.70, and labelled 39% of 143 normal flights as aircraft faults (Table 7). The signs fired at similar rates whichever party the uploader blamed (Fig. 2): motor saturation in 23% of aircraft-caused and 29% of pilot-caused crashes (1% of normal flights), battery warnings in 15% and 18% (3%). Error messages and failsafes were no more frequent in crashes than in normal flights. Two signals noticed after the development run did not replicate on held-out crashes: accelerometer clipping (24% against 8%, $p = 0.045$) and manual mode at the crash (18% against 10%, $p = 0.165$).

†*Independent relabelling.* To test whether the failure lies in the uploader labels, a reviewer who is not an author relabelled 72 modelled crashes from plots alone (all 36 pilot-rated and 36 randomly drawn aircraft-rated crashes; no log ID, date, notes, rating or Flight Review link; scoring declared in `fa5fb0b` before the pack was built). The reviewer called 30 crashes aircraft-caused, 28 pilot-caused, 13 environmental and 1 undecidable. On the 58 decided as pilot or aircraft, agreement with the uploader was no better than chance (45%, $\kappa = -0.10$, -0.34 to 0.17): 16 of 36 pilot-rated crashes were called aircraft-caused (44%, Wilson 30–60%) and 16 of 36 aircraft-rated crashes pilot-caused. Re-scored against the reviewer's labels, the declared rule reached a balanced accuracy of 0.55 [0.42, 0.66] (0.36 against the uploader labels on the same cases), FlightMD 0.52 [0.41, 0.64], and "tilt event present" an AUC of 0.54 [0.41, 0.66]. By the declared wording rule, the attribution failure is therefore not explained by uploader labels: with independent labels the tools still did not separate the causes. The two sets of labels also disagree with each other, so neither should be treated as ground truth.

In Task B, sample sizes decline across margins as flights under 10 s are excluded: 112 drone and 36 pilot crashes at $m = 0$ s ($n = 148$), 106 vs. 36 at $m = 2$ s ($n = 142$), 101 vs. 34 at $m = 5$ s ($n = 135$), 87 vs. 27 at $m = 10$ s ($n = 114$), 73 vs. 27 at $m = 20$ s ($n = 100$), and 63 vs. 26 at $m = 30$ s ($n = 89$). On the full sample ($n = 148$ at $m = 0$ s, 75.7% drone-blamed), cross-validated logistic regression (mean AUC over five repeats of stratified 5-fold CV, the declared leak-curve scheme) gave 0.49 at $m = 0$ s and 0.42–0.52 across cut-offs; on the constant 89-crash subset it gave 0.45–0.46. †Earlier versions reported pooled AUCs (0.408–0.501) with a bootstrap that did not refit the model, which understated the uncertainty. We therefore compared each observed AUC with a permutation null that refits the

full cross-validation for each of 200 label shuffles (clustering and refit audit, declared in `6fff27d`). The null SD was 0.063–0.085 for logistic regression and 0.044–0.065 for depth-3 trees. No cell had a permutation p below 0.05. Approximate upper bounds, observed AUC +1.96 null SD, were 0.58–0.65 for logistic regression and 0.54–0.69 for trees; they are approximate because the null SD measures the spread without a signal, not the standard error at the observed AUC. The best tree cell ($m = 5$ s, AUC 0.593, $p = 0.055$) is one of 24 comparisons, so the trees cannot rule out a modest signal. With 36 pilot-labelled crashes and the refit null SD at $m = 0$ s (0.080), 80% power requires an AUC of about 0.72 ($0.5 + 2.80 \times 0.080$). An operational attribution verdict would need an AUC of about 0.8 or higher, which no model approaches. Earlier tree values of 0.379 and 0.366 at $m = 10$ and 30 s scored predictions averaged across repeats; averaging per-repeat AUCs gives 0.463 and 0.430. Only the fixed rule was prospectively declared; logistic models, decision stumps and depth-3 trees are exploratory. †The rule did not improve as the crash was cut away. Re-running it at each cut needs the rule's signs computed from the leak-curve features, so this is a feature-based proxy of the declared rule, not the rule itself, and a crash with no sign counts as wrong. On the modelled crashes the proxy's balanced accuracy was 0.46 [0.38, 0.54] with the full window and 0.45 [0.37, 0.54], 0.39 [0.31, 0.50], 0.37 [0.28, 0.47] and 0.35 [0.25, 0.45] with the last 5, 10, 20 and 30 s removed (bootstrap over crashes; audit `9650a69`). It falls below 0.5 because an aircraft sign fires on 55–68% of all crashes, pilot-rated ones included, so only 19–29% of pilot-rated crashes are called pilot; undecided crashes are only 7–11%, and on decided crashes alone the figure is 0.50 [0.41, 0.59] at $m = 0$ s and 0.39 [0.28, 0.50] at $m = 30$ s. Values below 0.5 therefore reflect signs that fire at least as often on pilot-rated crashes, plus undecided crashes counted as wrong; they are not evidence of a reversed signal. †Against the same refitting null, stumps gave 0.447 at $m = 0$ s and 0.481 at $m = 10$ s (approximate upper bounds 0.54 and 0.59; $p = 0.77$ and 0.47), replacing earlier values (0.430, 0.413) computed on predictions averaged across repeats.

If an operator requires not merely statistical separation from chance but confirmation that performance excludes a mediocre baseline (AUC ≤ 0.80) via a 95% confidence lower bound under an operational target of 0.85 ($0.85 - 1.96 \times \mathrm{SE} \geq 0.80 \implies \mathrm{SE} \leq 0.0255$), the Hanley–McNeil standard error, $\mathrm{SE}^2 = [A(1-A) + (n_p - 1)(Q_1 - A^2) + (n_a - 1)(Q_2 - A^2)]/(n_p n_a)$ with $A = 0.85$, $Q_1 = A/(2-A)$, $Q_2 = 2A^2/(1+A)$ and the observed 3.1 aircraft crashes per pilot crash ($n_a = 3.1\, n_p$), meets this criterion at about $n_p = 100$

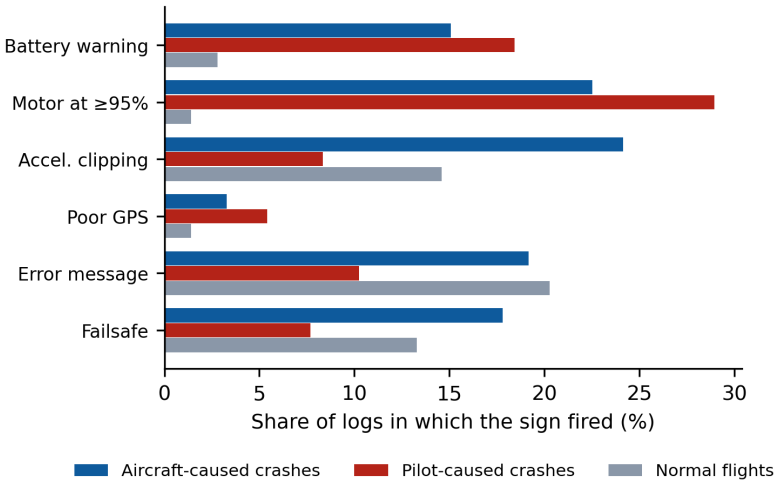


Figure 2: How often each aircraft-fault sign fired in held-out crashes, by uploader label, and in normal flights.

independently verified pilot-fault crashes (SE ≈ 0.025); 80% power to exclude 0.80 when the true AUC is 0.85 (SE $\leq 0.05/2.80 = 0.018$) needs about 200 (SE ≈ 0.018). Both are far beyond the 36 available in the modelling sample (SE 0.042 at $n_p = 36$). An earlier version stated 250, which applied the null-hypothesis variance at AUC 0.85. We therefore report that these rules, models and labels give no usable attribution signal under the evaluated setup, not that attribution is fundamentally impossible.

Table 7: Tests, their declared pass marks and outcomes. Holm adjustment over the seven p-valued tests. ‡First-pass analysis, not declared before running (Table 4).

Test (declared criterion)	Result	Holm p	Outcome
Protections disabled, crash vs normal‡	51% vs 46%, $p = 0.63$	1.00	No difference
Attribution rule, dev. (BA ≥ 0.70)	BA 0.57, 35% wrong	–	Failed
Attribution rule, held-out	BA 0.47, 47% wrong	–	Failed
Clipping, aircraft vs pilot ($p < 0.025$)	24% vs 8%, $p = 0.045$	0.27	Not supported
Manual mode, pilot vs aircraft ($p < 0.025$)	18% vs 10%, $p = 0.165$	0.73	Not supported
AUC drop from 0 to 10 s (≥ 0.10)	0.88 to 0.87 (stratified); 0.833 to 0.799 (grouped, one cohort)	–	Not supported
Within-drone vibration ($p < 0.025$)	$\bar{r} = 0.48$, $p = 0.67$	1.00	Not supported
Within-drone tracking ($p < 0.025$)	$\bar{r} = 0.50$, $p = 0.47$	1.00	Not supported
Between-drone vibration ($p < 0.025$)	$\bar{r} = 0.67$, $p = 0.0023$; †clustered $p = 0.012$ (airframe), 0.006 (operator site)	0.016; †0.08, 0.042	Supported as declared; suggestive once clustered by airframe or operator site (one set per site: $p = 0.14$); replication inconclusive
Between-drone tracking ($p < 0.025$)	$\bar{r} = 0.57$, $p = 0.15$	0.73	Not supported
Expert-fault checks, $m = 10$ s ($\geq 50\%$, $\leq 10\%$)	best: vibration 3/5 vs 11%	–	None met

5.3. The crash window (Q3a)

In the primary declared sample, crash flights were separated from normal flights with a cross-validated AUC of 0.88 (SD 0.01) at $m = 0\,\text{s}$ ($n = 148$ crashes, 139 normal flights), 0.88 at $m = 2\,\text{s}$ (142 vs. 137), 0.88 at $m = 5\,\text{s}$ (135 vs. 133), 0.87 (SD 0.01) at $m = 10\,\text{s}$ (114 vs. 130), 0.86 at $m = 20\,\text{s}$ (100 vs. 128), declining to 0.86 at $m = 30\,\text{s}$ (89 vs. 124); the declared drop of 0.10 was not observed (observed drop of 0.01). To evaluate whether this apparent stability was an artefact of sample attrition as shorter flights dropped out at larger margins, we evaluated the cross-validation on the fixed constant subset of 213 flights with duration $\geq 40\,\text{s}$ (89 crashes, 124 normal flights; plotted in Fig. 3) across all six margins. On this constant sample, model AUC declined smoothly from 0.895 (SD 0.007) at $m = 0\,\text{s}$ to 0.876 at $m = 2\,\text{s}$, 0.873 at $m = 5\,\text{s}$, 0.868 at $m = 10\,\text{s}$ (a drop of 0.027), 0.862 at $m = 20\,\text{s}$, and 0.864 at $m = 30\,\text{s}$. The absence of a large drop in Task A AUC (0.895 to 0.868) indicates that the cross-sectional model does not rely primarily on transient crash-impact telemetry. Instead, classification is likely influenced by flight envelope differences and airframe characteristics that correlate with duration (Table 9). Although flight duration was deliberately excluded from the 11 normalized model features, duration alone separates crash from normal flights with an AUC of 0.662 at $m = 0\,\text{s}$ (crash median 71.0 s vs. normal median 169.0 s) and 0.634 at $m = 10\,\text{s}$, and correlates significantly with several normalized features: longer flights exhibit larger 95th-percentile tilt ($\rho = +0.470$ at $m = 0\,\text{s}$, $+0.484$ at $m = 10\,\text{s}$), greater voltage sag ($\rho = +0.321$ and $+0.357$), and larger stick deflections ($\rho = +0.363$ and $+0.233$), while shorter flights show higher rate-tracking errors ($\rho = -0.248$ at $m = 0\,\text{s}$). The absence of a large collapse holds on identical flights, confirming that changing sample composition across cut-offs did not mask a collapse.

Leakage was nevertheless present for individual physical checks. The rate-tracking feature alone fell from an AUC of 0.81 to 0.69 as the crash was excluded, while vibration stayed between 0.72 and 0.77 (Table 9). Evaluating sign firing rates across margins on 148 crashes and 139 normal flights illustrates the physical mechanism: accelerometer clipping in crashes fell from 26.4% ($m = 0\,\text{s}$) to 16.7% ($m = 10\,\text{s}$, a 37% relative drop), while in normal flights it remained baseline at 4.3% $\rightarrow$ 3.8% (sign: more than 50 clipping counts before t_c; denominator: all 139 normal flights. Fig. 2 uses the held-out sample with only logs that record clipping in the denominator, 7 of 48 normal flights, 15%). Failsafe activation in crashes dropped from 13.5% to 8.8% at $m = 10\,\text{s}$ and 6.0% at $m = 20\,\text{s}$, while in normal flights it remained steady at

$\sim$ 13–14% (standard return-to-land and disarm triggers). Motor saturation in crashes declined from 45.9% to 41.2% (and in normal flights from 18.7% to 14.6%), reflecting exposure window truncation alongside impact saturation. A fixed tuning check that found 10 of 12 candidate expert-diagnosed tuning faults (7 of 9 on paired surviving logs) with the crash included found only 1 of 9 once the last 10 s were removed (Section 5.10).

Crucially, the high cross-sectional model separation (AUC 0.88) does not demonstrate unconfounded anticipation of an impending crash: crash and normal flights differ fundamentally in duration (median 54 s against 388 s in the broader archive; 71 s vs. 169 s in the complete-case sample), firmware era distribution, and operational flight envelope. Table 9 gives the single-feature AUCs. Values below 0.5 indicate features that are lower in crash flights: crash flights reached lower tilt and smaller stick deflections over the whole flight than normal flights, consistent with short, aborted or cautious testing flights. Table 8 reports a step-by-step decomposition, regenerated with fixed seeds[†]. In this sample crash and normal flights never share a vehicle (the 39 normal flights removed in the factorial audit share vehicles with the separate within-drone crashes, not with these 148), but 185 of the 287 flights come from 34 vehicles with several flights each (one contributes 39), so ungrouped folds let the model recognise individual vehicles within a class. Grouping folds by vehicle (Step 1) lowers the AUC from 0.882 to 0.826; an earlier version reported 0.884 for this step, which we could not reproduce with vehicle identifiers. On the constant cohort of 213 flights, grouped folds give 0.833 at $m = 0$ s (Step 1b), so removing the final 10 s or 30 s (Steps 2 and 3) lowers AUC only to 0.799 and 0.791; with grouped folds at all six cut-offs on this cohort the drop from $m = 0$ to 10 s is 0.035, so the declared leak criterion still fails. Removing tilt and stick features (Step 4) lowers the AUC to 0.738. An earlier exploratory step scored the 67 multi-flight crashes against other-aircraft normals and reported AUC 0.522 [0.413, 0.615]; that comparison mixed feature extractors and cut-offs between crash and control flights and is superseded by the factorial audit below, where the same crashes against matched other-aircraft flights under one extractor give 0.623 [0.523, 0.716] (Table 8, Step 4b). The 67 within-drone crashes are not a subset of the 103 multi-history crashes: only 10 of the 67 are in the 148-crash sample. To test whether the 0.88 cross-sectional separation was an artifact carried solely by single-history crashes (vehicles with no earlier flights in the archive: maiden flights, bench tests), we scored single-history crashes ($n = 45$) and multi-history crashes ($n = 103$) from the Step 0 sample separately against

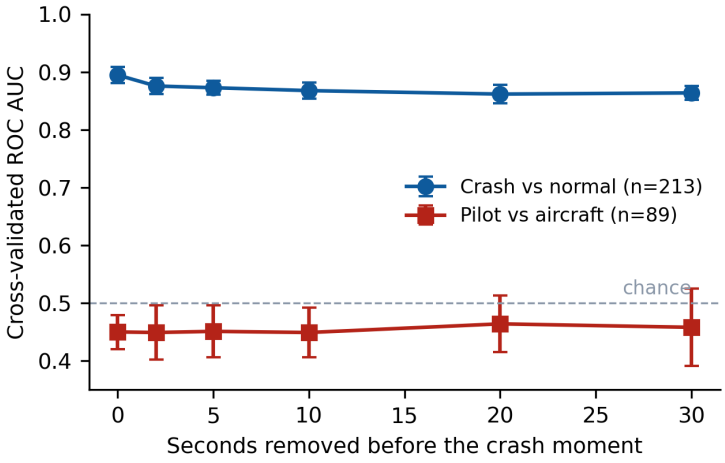


Figure 3: Cross-validated AUC on the constant subset ($n = 213$ for crash vs. normal, $n = 89$ for pilot vs. aircraft) as the end of the analysis window moves away from the crash moment. Bars: 95% bootstrap confidence intervals.

the 139 normal flights. Both subsets yielded high cross-sectional separation: single-history crashes reached AUC 0.835 ± 0.010 ($\pm$ is the SD over five CV repeats here and below) at $m = 0\,\mathrm{s}$ (0.801 ± 0.020 at $m = 10\,\mathrm{s}$), while multi-history crashes reached AUC 0.891 ± 0.007 (0.884 ± 0.010 at $m = 10\,\mathrm{s}$). This indicates that the 0.88 headline AUC is not an artifact confined to maiden flights; rather, cross-sectional separation may reflect duration and flight-envelope differences across the archive (window duration alone gives AUC 0.662).

Table 8: Decomposition of model separation between crash and normal flights. †All values regenerated from committed scripts with fixed seeds (clustering and refit audit, `6fff27d`: seeds 42–46, clusters sorted before fold assignment, 95% cluster bootstrap over vehicles with 2,000 resamples; Step 4b from the factorial audit). Earlier, not reproducible values were 0.883, 0.884, 0.860, 0.874, 0.831, 0.529 and 0.569 for Steps 0–6.

Step	Evaluation protocol specification	Sample flights	ROC AUC [95% CI]
0	Cross-sectional: stratified 5-fold CV ($m = 0$ s)	148 crash, 139 normal	0.882 [0.776, 0.935]
1	Folds grouped by vehicle	148 crash, 139 normal	0.826 [0.726, 0.895]
1b	Same, constant cohort ($m = 0$ s)†	89 crash, 124 normal	0.833 [0.730, 0.914]
2	Crash window removed ($m = 10$ s), constant cohort, grouped	89 crash, 124 normal	0.799 [0.668, 0.891]
3	Late window removed ($m = 30$ s), constant cohort, grouped	89 crash, 124 normal	0.791 [0.662, 0.889]
4	Without tilt and stick features ($m = 0$ s), grouped	148 crash, 139 normal	0.738 [0.648, 0.827]
4b	67 within-drone crashes vs. matched other-aircraft flights, one extractor ($m = 10$ s, grouped)	67 crash, 311 normal	0.623 [0.523, 0.716]
5	67 within-drone crashes vs. own earlier flights, stratified	67 crash, 305 earlier	0.546 [0.485, 0.605]
6	Same, folds grouped by drone	67 crash, 305 earlier	0.559 [0.512, 0.617]
6b	Step 6 without earlier flights flown after the crash (GPS time)	57 crash, 227 earlier	0.582 [0.524, 0.658]

Step 4b supersedes an earlier value of 0.522 that mixed feature extractors between crash and control flights. The paired comparison of other-aircraft and own earlier flights uses one pipeline (recomputed features, $m = 10$ s, full window, grouped) on the same 67 crashes: Step 4b's 0.623 against 0.539 for own earlier flights (not Step 6's 0.559, which uses stored features) gives +0.083 [−0.007, 0.166]; without post-crash flights the own-flight cell is 0.510 and the difference +0.113 [0.029, 0.212]. The 0.507 quoted elsewhere is the same comparison on the factorial audit's constant subset (Section 5.5).

Table 9: Single-feature AUC for separating crash from normal flights at three cut-offs (no fitting; 0.5 is chance; values below 0.5 mean lower in crash flights).

Feature	$m = 0\,\mathrm{s}$	$m = 10\,\mathrm{s}$	$m = 30\,\mathrm{s}$
Rate-tracking error	0.81	0.72	0.69
Vibration (p95)	0.74	0.72	0.77
Manual share	0.67	0.68	0.69
Motor saturation	0.67	0.66	0.65
Clipping rate	0.59	0.54	0.55
Voltage sag	0.55	0.53	0.48
Poor GPS	0.53	0.55	0.54
Failsafe	0.50	0.47	0.47
Battery minimum	0.42	0.40	0.38
Tilt (p95)	0.41	0.36	0.33
Stick deflection (p95)	0.32	0.33	0.35

5.4. Crash moments and archive labels (exploratory)

Crashes with a confirmed crash moment†. The crash moment t_c is defined by tilt above 60°; we call a crash confirmed when that tilt is also followed within 10 s by disarming or a landed or ground-contact flag. Of the 148 modelled crash logs, 72 are tilt-defined (40 confirmed, 32 not) and in 76 tilt never exceeds 60°, so t_c falls back to the end of the armed period. By index vehicle type (Section 4.2) the 148 are 108 multicopters, 18 fixed-wing and 22 VTOL logs. Tilt-defined crashes are 41 of the 108 multicopters (29 confirmed), but 15 of the 18 fixed-wing (2 confirmed) and 16 of the 22 VTOL logs (9 confirmed). For fixed-wing and VTOL aircraft a bank above 60° can occur in normal flight, and an aircraft that banks and then lands and disarms meets the "confirmed" condition without crashing, so the tilt rule is not a crash criterion for them. †We therefore restrict crash-moment results to multicopters and report the mixed set only for comparison. Among the 108 multicopters, 41 (38%) are tilt-defined, the same share as in the rest of the archive (39%; Section 5.4).

†Two authors then judged, by eye, the traces of 50 randomly drawn crash logs (tilt and altitude around t_c). The first labelled a sheet that showed the automatic verdict (declared check `97974ad`); the second labelled the same logs on a blind sheet, in a new order without identifiers or verdicts (`59c8212`). They agreed on whether the computed crash moment was right in 40 of 47 decidable logs (85%, Cohen's $\kappa = 0.70$, bootstrap 95% CI 0.50–0.87). Of the 26 tilt-defined crash moments, the reviewers judged 21 and 24 correct, including 11 and 12 of the 12 confirmed ones; of the 24 fallback crash moments, only 2 and 6. †By vehicle type (34 multicopter, 7 fixed-wing and 9 VTOL logs), both reviewers judged all 15 multicopter tilt-defined crash moments correct (10 confirmed, 5 not); the disagreements on tilt-defined moments were all in fixed-wing (4 and 4 of 5 correct) and VTOL logs (2 and 5 of 6). Of the 19 multicopter fallback moments, 2 and 5 were judged correct. †A third rater who is not an author then labelled the same blind sheet (scoring declared in `fdb3b7c` before the sheet was sent). On whether the computed crash moment is right, the three agreed well (Fleiss' $\kappa = 0.71$, 0.53–0.86); the third rater judged 24 of 26 tilt-defined moments correct (all 14 unconfirmed and 10 of 12 confirmed) and 2 of 24 fallback moments. Whether fallback logs contain the crash elsewhere was disputed ($\kappa = 0.32$, 0.10–0.57): the first reviewer saw no crash in 18 of 24, while the blind reviewer saw a crash before the computed moment in most of them, one that never crossed 60° tilt. †We then tried an objective event detector, the earlier of tilt above 60° and an accelerometer

reading above 4 g (declared audit `35103da`). It agreed with the first reviewer on whether a log contains the crash ($\kappa = 0.72$, 0.49–0.91) but not with the blind reviewer ($\kappa = 0.38$, 0.12–0.65), and it fired on 89 of 139 normal flights (hard landings), so it failed its declared validation gate and we do not use it to redefine t_c. †A second detector, declared after that failure (`9650a69`), combined four cues: tilt above 60°; a descent faster than 5 m/s ending within 3 s in ground contact or disarm; a disarm while the land detector still reported flight; and an acceleration above 6 g within 2 s before landing or disarm. It again matched the first reviewer ($\kappa = 0.82$, 0.62–0.96) but not the blind one ($\kappa = 0.32$, 0.11–0.57), and fired on 45 of 139 normal flights (32%; 40 through the impact cue), so it also failed the gate (both $\kappa \geq 0.6$, false alarms $\leq 10\%$). The reviewers disagree mainly on low-tilt logs, so a detector tuned to either cannot be validated by both; the third rater (below) did not remove that disagreement.

†Because the tilt-defined crash moment on multicopters is the one both reviewers support, we repeated the window analyses on tilt-defined crashes (Table 10). These subgroups are post hoc: the tilt-defined set was chosen after the confirmed-crash result (Appendix Appendix A), and the multicopter restriction after the review above. With grouped folds on the constant cohort (124 normal flights; intervals from a paired vehicle bootstrap, declared audit `9650a69`), removing the last 10 s lowered AUC by 0.035 [0.005, 0.077] over all 89 crashes but by 0.114 [0.027, 0.222] over the 24 tilt-defined multicopter crashes (0.878 to 0.764). That meets the declared leak criterion (≥ 0.10) by its point estimate only; the interval includes 0.10. In the factorial pipeline, the mean $m = 0$ versus 10 s difference over the four crash and normal sets was 0.014 [−0.027, 0.038] for all crashes and 0.089 [0.019, 0.168] for tilt-defined multicopter crashes (21 and 16 in the two crash sets), which meets the declared materiality rule (0.05 with an interval excluding zero).

†Because the cut removes the very tilt that defines the subgroup, part of this effect could be mechanical. We refitted without the tilt and stick features (declared before the run, with the wording rule that the abstract would drop "material" if the effect fell below 0.05). The effect grew rather than shrank: 0.216 [0.086, 0.402] on the leak curve and 0.131 [0.042, 0.226] in the factorial pipeline, because without those features AUC after the cut falls further (0.583 against 0.764 for the full model on the leak curve). The last seconds before a tilt-defined crash therefore carry signal in several features, not only the defining tilt. Crash-window leakage is small overall but material among multicopter crashes whose log shows a crash-like tilt; the small overall

Table 10: †Crash-window effect by crash subset (exploratory, post hoc). Leak drop: AUC at $m = 0$ minus AUC at $m = 10$ s, grouped folds, constant cohort with 124 normal flights. Factorial: mean $m = 0$ minus 10 s difference over the four crash and normal sets. 95% intervals from a paired vehicle bootstrap. "No tilt/stick": refitted without tilt and stick features.

Crash subset	n	Leak drop	No tilt/stick
All crashes	89	0.035 [0.005, 0.077]	0.063 [0.023, 0.109]
Tilt-defined, all types	43	0.084 [0.018, 0.139]	0.115 [0.044, 0.188]
Confirmed, all types	25	0.103 [0.027, 0.188]	0.125 [0.013, 0.247]
Tilt-defined multicopters	24	0.114 [0.027, 0.222]	0.216 [0.086, 0.402]
Confirmed multicopters	18	0.168 [0.048, 0.340]	0.155 [−0.032, 0.491]
Factorial pipeline			
All crashes	145	0.014 [−0.027, 0.038]	–
Tilt-defined, all types	63	0.067 [0.015, 0.131]	–
Tilt-defined multicopters	37	0.089 [0.019, 0.168]	0.131 [0.042, 0.226]

figure is possibly in part due to the many logs without one. The two figures of 0.014 in this paper are different quantities that coincide after rounding: the Shapley share of the window cut (0.0144 [0.007, 0.023]; Section 5.5) averages the cut over every combination of the other four factors on the constant subset, with the models fixed in each bootstrap, so its interval is narrow; the four-cell difference (0.0136 [−0.027, 0.038]) compares just four separately fitted pairs on the full sets.

Within the same drone, tilt-defined crashes still showed no warning (vibration rank 0.47, $p = 0.66$, 18 drones; tracking 0.48, $p = 0.58$, 16 drones; model 0.55 [0.43, 0.67]).

How many crash-rated logs contain the crash†. To see whether this holds beyond our samples, we downloaded all 520 crash-rated logs from real hardware in the archive (declared audit `09cf99c`, amended in `18b0eff` to define multicopters by the index vehicle type, because older logs lack the rotary-wing flag). Of the 381 multicopter logs, 40 could not be read. Of the remaining 341, 132 (39%) show tilt above 60° (91 confirmed) and 209 do not. So in 61% of the archive's crash-rated multicopter logs the standard tilt rule finds no crash, and the computed crash moment there is usually wrong. How many of those logs contain the crash at all depends on the reviewer: combining the archive shares with each reviewer's labels gives 50% [40–62%] of crash-rated

multicopter logs containing the crash for the first reviewer and 86% [75–97%] for the blind reviewer (66% [47–86%] on the logs where they agree), so we do not claim a single figure. †The third rater sided with the blind reviewer on 11 of the 13 logs where the first two disagreed, and the majority of the three gives 80% [68–92%]; but agreement among all three on this question stayed low (Fleiss' $\kappa = 0.33$, 0.05–0.58), so by the declared rule we still report the range rather than one figure. The share differs by label: 110 of 247 aircraft-rated logs (45%) but 22 of 94 pilot-rated logs (23%) show a tilt event (two-sided Fisher exact $p = 0.0003$), and it is lower in recent logs (43% up to 2019, 44% in 2020–2022, 23% from 2023). †Pilot ratings are also more common in recent logs (46 of the 81 readable multicopter logs from 2023 onwards are pilot-rated), so we adjusted for era and vehicle type (declared audit `35103da`): aircraft-rated logs remained about twice as likely to show a tilt event (Mantel–Haenszel odds ratio 2.08 [1.17, 3.69], $p = 0.011$; logistic regression gives the same). On its own, "tilt event present" would separate aircraft-rated from pilot-rated logs at AUC 0.61 [0.55, 0.66] across the archive but only 0.55 [0.45, 0.64] in our 148 modelled crashes and 0.53 [0.43, 0.63] in the 108 modelled multicopters, consistent with our classifiers staying at chance. Our modelled sample matches the rest of the archive (38% against 39%, $p = 0.91$). The uploader's label therefore partly encodes whether the log shows a tilt-type crash. Two readings fit this: the recording may stop before the crash more often in pilot-rated logs, or pilot-caused crashes may physically involve less tilt (for example a controlled descent into an obstacle rather than a loss of attitude control); the reviewers' disagreement on low-tilt logs means we cannot tell them apart. Either way, a classifier could appear to separate the labels in this archive by detecting tilt-type crashes alone. †To check whether tilt hides a real signal, we repeated the attribution model within each tilt stratum of the modelled multicopters (declared audit `9650a69`; 200 refitting permutations): AUC 0.63 among 41 tilt-defined crashes (9 pilot-rated; $p = 0.10$; approximate bounds 0.39–0.87) and 0.62 among 67 without tilt (18 pilot-rated; $p = 0.11$; 0.43–0.81). Neither is distinguishable from chance, but with 9 and 18 pilot-rated crashes neither rules out a usable signal either.

5.5. Factorial optimism audit (exploratory)

†The steps of Table 8 change several things at once, and their samples were built with different feature extractors. To find what the crash-vs-normal AUC depends on, we declared a factorial audit after the main analyses (commit

`3c34530`; amended `f6ba424` to a float32 tolerance after its extractor check aborted on rounding differences; run 27 October 2026). Every flight was re-extracted from the raw log by one extractor, and five two-level factors were crossed: crash set (103 multi-history crashes vs. the 67 within-drone crashes), comparison flights (uploader-rated good/great normals vs. unrated flights from drones with no crash-rated log, matched to the within-drone cases on hardware, PX4 version and vehicle type), crash-window cut ($m = 0$ vs. 10 s), fold scheme (stratified by flight vs. grouped by vehicle) and window (full flight vs. first 30 s airborne). Normal flights from any crashed vehicle were removed (39 of the 139). Intervals come from one paired two-stage cluster bootstrap (2,000 resamples over crash and control vehicles, models not refitted); a factor was declared material if its share was ≥ 0.05 with a 95% interval excluding zero. The declaration passed all abort checks (anchors 0.891 and 0.559; extractor agreement 374/374; label-permutation mean 0.495).

The declared primary 2×2 ($m = 10$ s, grouped folds, full window) did not isolate a cause (Table 11): changing the normal flights lowered AUC by 0.094 [−0.014, 0.197] and changing the crash set by 0.108 [−0.037, 0.224]. Following the pre-declared wording, the AUC depends on the comparison set; the cause is not isolated. On the constant subset (63 and 60 crashes), AUC fell from 0.878 at the most optimistic setting to 0.666 at the most conservative, a drop of 0.212 [0.062, 0.335]. An exact Shapley split assigned 0.062 [0.034, 0.087] of it to vehicle-grouped folds, the only material factor; the effect was concentrated where comparison drones contribute several flights (interaction with comparison set +0.082 [0.006, 0.122]), consistent with the regenerated Step 1 of Table 8, where grouping by vehicle lowered the AUC from 0.882 to 0.826. The crash-window cut carried a small but non-zero Shapley share of 0.014 [0.007, 0.023] (on all crashes; Section 5.4 gives the larger effect on tilt-defined crashes), and the fixed 30 s window carried 0.001. Separation against other aircraft persisted at the most conservative setting (0.666 [0.557, 0.770]); replacing other-aircraft flights by the same drones' earlier flights at the same settings (first 30 s window, $m = 10$ s, grouped folds) lowered AUC by a further 0.21, from 0.666 to 0.455. [†]On all 67 within-drone crashes with the full window, the paired difference between other-aircraft and own earlier flights is +0.083 [−0.007, 0.166] (0.623 against 0.539), and +0.113 [0.029, 0.212] once earlier flights flown after the crash are removed (0.623 against 0.510, both from the factorial pipeline's same-aircraft cell; clustering and refit audit). A fixed 30 s window also did not reproduce the caliper lift within drones (Group-CV 0.455 [0.377, 0.507]); because that window also removes

Table 11: †Factorial optimism audit. Top: declared primary 2×2 (103 multi-history and 67 within-drone crashes) ($m = 10$ s, vehicle-grouped folds, full window). Bottom: exact Shapley split of the 0.212 [0.062, 0.335] AUC drop from the most optimistic to the most conservative other-aircraft setting (constant subset, 63/60 crashes). 95% intervals from a paired two-stage cluster bootstrap.

Crashes vs. comparison flights	Crashes	Comparison	AUC [95% CI]
103 vs. rated normals	78	91	0.825 [0.699, 0.908]
103 vs. matched unrated	78	311	0.686 [0.591, 0.777]
67 vs. rated normals	67	91	0.673 [0.540, 0.838]
67 vs. matched unrated	67	311	0.623 [0.523, 0.716]
Factor (optimistic → conservative)	**Shapley share of drop**		**Material**
Folds: stratified → by vehicle	0.062 [0.034, 0.087]		Yes
Comparison: rated → unrated	0.070 [−0.033, 0.157]		No
Crash set: 103 → 67	0.064 [−0.062, 0.186]		No
Crash window: $m = 0 \rightarrow 10$ s	0.014 [0.007, 0.023]		No (< 0.05)
Window: full → first 30 s	0.001 [−0.031, 0.034]		No

proximity to the crash, it is not a clean test of the masking explanation. One same-drone cell with flight-stratified folds fell below chance (0.393), which we attribute to splitting one drone's flights across folds and do not interpret.

5.6. Within the same drone and between drones (Q3b, Q3c)

Within-drone model and duration matching. On the 67 multi-flight drones with 372 flights (67 pre-crash flights cut 10 s before impact and 305 earlier flights), three complementary statistics evaluate within-drone precursor separation. Evaluated across flights, stratified folds give 0.546 [0.485, 0.605] (Step 5) and folds grouped by drone 0.559 [0.512, 0.617] (Step 6), regenerated with fixed seeds; an earlier, non-reproducible run gave 0.569 with a cluster interval of [0.500, 0.643]. †After removing the 85 earlier flights that GPS time shows were flown after the crash (Section 5.6; declared audit `0d38f5d`), Step 6 gives 0.582 [0.524, 0.658] on 57 drones (Step 6b). The same comparison on recomputed features (the factorial audit's same-aircraft cell) gives 0.507 [0.436, 0.582]. We therefore describe the within-drone model result as at most a weak signal that depends on the feature pipeline. The mean within-drone pairwise AUC (0.592 [0.516, 0.692]) evaluates ranking solely within each drone's flight history; its confidence interval extends down to near-chance (0.516), consistent with the non-parametric within-drone rank test on individual physical

features ($\bar{r} = 0.48, p = 0.67$ for vibration; $\bar{r} = 0.50, p = 0.47$ for tracking error), neither of which exceeds chance (0.50). Furthermore, feature-window duration (take-off to $t_c - 10\,\mathrm{s}$ for crashes; take-off to $t_{\mathrm{end}} - 10\,\mathrm{s}$ for earlier flights) alone separates earlier from crash flights with an effective AUC of 0.682 [0.634, 0.729] because earlier flights had an index requirement of $\geq 30\,\mathrm{s}$ while crash flights were included if $\geq 5\,\mathrm{s}$. Evaluating within-drone Group-CV across calipers ($\epsilon = |\Delta \ln(\text{duration})| \in [0.10, 1.00]$) reveals a smooth gradient: at $\epsilon = 0.10$ ($n = 27$ airframes), matched AUC is 0.580; at $\epsilon = 0.20$ ($n = 34$), 0.590; at $\epsilon = 0.25$ ($n = 36$), 0.604 (5-seed average 0.642 vs. 0.532 unmatched, lift $\Delta = +0.110$; †a declared rerun with reproducible folds gave 0.636 vs. 0.526, lift +0.110, paired cluster bootstrap 95% CI [+0.028, +0.205], replacing an earlier interval of [+0.074, +0.238] computed on a fold draw that cannot be reproduced); at $\epsilon = 0.50$ ($n = 45$), 0.596; rising to parity with unmatched at $\epsilon = 1.00$ (0.568 vs. 0.564). Univariate feature evaluation on the 36 airframes showed that the caliper gain is carried primarily by accelerometer clipping (+0.068), tilt envelope (+0.037), and voltage sag (+0.031). †Without these three features the lift is +0.077 [+0.013, +0.204] (5 seeds; seed 42 alone +0.062 [−0.008, +0.132]) and the matched within-drone AUC is 0.573. This meets the declared criterion for the reading that duration differences mask within-drone precursors (lift ≥ 0.05, interval excluding zero), but the lift is smaller and less stable than with all features, and a fixed 30 s window did not reproduce it (§5.5). †The caliper sample also contained flights flown after the crash. Without them 24 airframes remain, and the lift is +0.086 [−0.064, +0.184] with all features and +0.063 [−0.087, +0.159] without the three duration-linked features. Under its declared criterion the masking reading loses support. The lift is no longer clearly distinguishable from zero (+0.086 [−0.064, +0.184], 24 airframes); the interval still includes the earlier estimate, and with 24 airframes only a lift of about 0.18 could be detected, so the result is inconclusive rather than null. We withdraw the claim that duration differences mask precursors.

Relative to the same drone's earlier flights, the crash flight was neither more vibrating ($\bar{r} = 0.48$, 95% CI 0.38–0.58, $p = 0.67$, 61 drones) nor worse at tracking its commands ($\bar{r} = 0.50$, 0.38–0.63, $p = 0.47$, 46 drones; Fig. 4). Presented as an exclusion bound, the upper 95% confidence bound of 0.58 indicates absence of substantial pre-crash vibration surges within the same drone of the magnitude observed between drones ($\bar{r} = 0.67$). Relative to matched drones without a recorded crash, drones that later crashed vibrated more on their ordinary pre-crash flights ($\bar{r} = 0.67$, 0.57–0.77, $p = 0.0023$,

60 sets; Holm-adjusted $p = 0.016$), while the tracking error did not differ significantly ($\bar{r} = 0.57$, 0.44–0.70, $p = 0.15$, 47 sets). †The vibration p-value treats the 60 sets as independent, but they come from 40 crashed airframes (11 crashed more than once) and reuse 17 controls. A two-stage bootstrap that resamples crashed and control airframes, counting a drone drawn twice twice, gives mean rank 0.66 [0.50, 0.81]; its null-shifted one-sided p (the share of resamples at or above $2\bar{r} - 0.5$) is 0.012, or 0.08 after Holm adjustment over the seven tests, and the percentile method (share of resamples at or below 0.5) gives 0.025. Keeping one set per crashed airframe (1,000 random choices of 40 independent sets) gives median mean rank 0.66 and median exact $p = 0.012$, with $p < 0.025$ in 79% of choices. The rule declared before this analysis (commit `6fff27d`) called the result supported only if the clustered p was below 0.025 and its Holm-adjusted value below 0.05; it fails the second part, so we describe the result as suggestive rather than supported (an earlier two-stage interval, [0.536, 0.789], counted each resampled drone only once). †*Operator clustering.* Controls from the same operator are not independent, and the archive records no operator. We used each airframe's first GPS fix as a proxy and linked airframes within 2 km of each other (declared `cfb79b9`; coordinates were used in memory only). The 40 crashed airframes came from 23 sites, and in 5 sets the crashed drone and a control shared a site. Resampling sites rather than airframes gave a mean rank of 0.66 [0.51, 0.79] and $p = 0.006$ (Holm-adjusted 0.042); removing the 5 shared-site sets gave 0.67 ($p = 0.003$), and radii of 0.5 and 10 km gave $p = 0.010$ and 0.004. The declared wording rule would therefore call the result supported. We do not apply it: taking one set per site, which weights each operator once, gave a median mean rank of 0.61 and a median p of 0.14 ($p < 0.025$ in 2% of 1,000 choices), so the effect rests partly on a few sites with several crashed drones. We keep "suggestive" and report this as a deviation from the declared rule. The vibration difference was clearest for aircraft-caused crashes ($\bar{r} = 0.67$, $p = 0.005$, 46 sets) and similar in size but not significant for pilot-caused ones ($\bar{r} = 0.64$, $p = 0.16$, 14 sets).

As a screening signal, the crashed drone was the roughest of its matched set in 33 of 60 sets (55.0%, Wilson 95% CI 42.5–66.9%, against 40.3% expected under the exact Poisson-binomial null; exact one-sided Poisson-binomial $p = 0.0137$, which treats the sets as independent; †with one set per crashed airframe the median share is 55% and the median p 0.031). Furthermore, in 38 of 60 sets (63.3%, Wilson 95% CI 50.7–74.4%), the crashed drone exceeded the third quartile (Q3) of its matched controls. Sensitivity differed

between measurement sources: 32 of 48 crashed drones (66.7%, Wilson 95% CI 52.5–78.3%) exceeded control Q3 on the older estimator metric, compared to 6 of 12 (50.0%, Wilson 95% CI 25.4–74.6%) on the newer accelerometer metric.

In practical fleet operations, positive predictive value (PPV) depends heavily on the operational flag rate and the fleet base rate. In the archive, 284 distinct vehicles accounted for all 520 real-hardware crash-rated logs out of 38,320 distinct vehicle identifiers, establishing an empirical vehicle crash base rate of 0.741% (284/38,320). When restricted to screenable multi-flight vehicles (airframes with ≥ 2 recorded flights, the operational prerequisite for pre-crash longitudinal screening), the crash base rate is 0.689% (182 crashed vehicles out of 26,421 multi-flight airframes). We emphasize that these base rates represent the lifetime probability of an airframe ever receiving a crash rating in the archive index, rather than an operational flight-hour incident rate, because the public archive does not record flight hours per airframe. Reported confidence intervals for PPV reflect sampling uncertainty in sensitivity (derived via 10,000 bootstrap resamples over crashed cases); the operational base rate is treated as a fixed fleet parameter. We evaluate screening performance under two distinct operational paradigms:

Empirical matched-set screening (45.1% flag rate): In our matched sets, flagging drones that exceed the upper quartile of their matched controls yields 63.3% sensitivity (38 of 60). Because sets contain 1 or 2 controls, the weighted null flag rate is 45.1%. At the 0.741% vehicle base rate, this yields an operational PPV of 1.04% (95% bootstrap CI 0.83–1.25%), an unflagged risk of 0.496%, a relative risk of 2.09× over unflagged airframes, a lift of 1.40× over base rate, and 96.4 inspections per detected crash (PPV 0.96%, 104 inspections per crash at the 0.689% screenable base rate).

Fleet-wide reference screening, measured†*:* Earlier versions ranked each crashed drone against the 107 matched controls and *assumed* that a top-decile threshold flags 10% of non-crash drones. We rebuilt the reference from the archive (declared `4961234` before any reference log was downloaded): every vehicle with no crash-rated log and at least three flights of 30 s or more in each of the 31 strata of the 60 sets (hardware, PX4 version and vehicle type, plus measurement source), at most 40 per stratum, three flights each; 412 of 467 drones gave a usable value. Thresholds were fixed on a random half of the reference drones in each stratum and the flag rate was *measured* on the other half, then the halves were swapped; intervals come from 2,000 bootstrap resamples of crashed and reference drones with thresholds refitted. Against

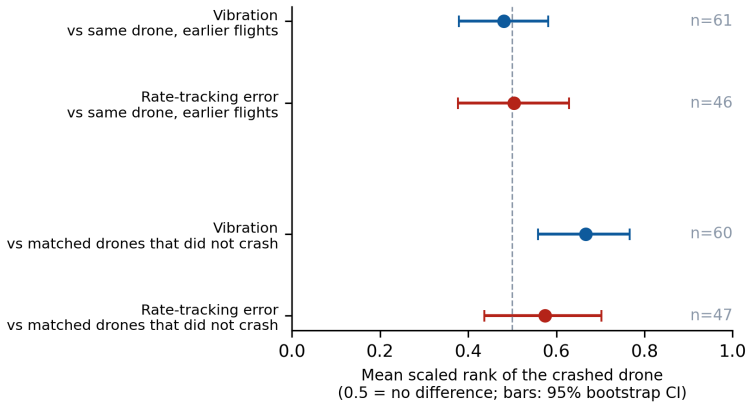


Figure 4: Mean scaled rank of the crashed drone against its own earlier flights (top) and against matched drones without a recorded crash (bottom). 0.5 indicates no difference.

this reference the crashed drones had AUC 0.65 [0.57, 0.74]. A threshold set to flag 10% of reference drones flagged 13.7% [13.4, 19.8] of unseen ones and caught 24% [17, 40] of the crashed drones, a lift of 1.77× [1.00, 2.60] over the 0.741% base rate (PPV 1.3%, about 76 inspections per crash); the nominal 25% and 5% thresholds flagged 27.0% and 11.3% and gave lifts of 1.46× [1.10, 2.14] and 1.66× [0.93, 2.83]. Because the interval at the 10% operating point reaches 1, by the declared rule vibration ranking is *not shown to beat random inspection*. The earlier figures (PPV 2.26–2.43%, lift 3.3× at an assumed 10% flag rate) overstated the screening value and are withdrawn. In 13–15 of the 31 strata fewer than four reference drones were available and source-level pooled thresholds were used, as declared; 143 of the 1,401 reference logs gave no vibration value. Vibration ranking is therefore at most a weak inspection aid, not a crash warning.

Flights flown after the crash in the within-drone samples†. The within-drone selection kept up to five earlier flights per crash that were dated on or before the crash day (475 flights for 101 crashes); 305 of them had usable features in the 67 analysed drones. A declared audit (commit `0d38f5d`) compared GPS UTC start times for the 217 crash-day entries (210 distinct flights, because a drone that crashed more than once lists some flights twice): 107 preceded the crash, 85 followed it and 18 had no usable GPS time; 29 crashes had at least one following flight. Removing the following flights removed 78 of the

305 analysed flights (the other 7 had no usable features). Without them, the within-drone rank tests still showed no warning (vibration $\bar{r} = 0.488$, $p = 0.60$, 44 drones; rate tracking $\bar{r} = 0.551$, $p = 0.21$, 30 drones); the within-drone model gave 0.582 [0.525, 0.660] on stored features (57 drones, 227 earlier flights) but 0.507 [0.436, 0.582] on recomputed features; and the duration-caliper lift (Section 5.6) was no longer clearly distinguishable from zero. Removing the 18 undated flights as well changed none of these conclusions (`postcrash_audit.md`).

5.7. Robustness of the between-drone result (exploratory)

The vibration result held when controls were restricted to the same airframe as the case ($\bar{r} = 0.69$, $p = 0.008$, 39 sets), when case flights from the crash day were removed ($\bar{r} = 0.76$, $p = 0.003$, 25 sets) and after adjusting for flight duration ($\bar{r} = 0.69$, $p = 0.0005$, 57 sets; Fig. 5). Duration was barely correlated with vibration (Spearman $\rho = 0.05$ and -0.04 for the two measurement sources), although case flights were longer than control flights (median 289 s against 173 s). The sharp sample shrinkage when removing crash-day flights (from 60 to 25 sets) indicates that many pre-crash flights in the public archive occurred during the same flight session or day. The archive index did not record intra-day flight sequence. †An earlier check of this ordering compared calendar dates only and could not detect a same-day flight flown after the crash. A declared check of GPS UTC start times (flight-order audit, commit `0e26db0`, amended `187f570`) found that of 95 distinct crash-day exposure flights in 39 of the 60 sets, 38 preceded the crash, 47 followed it and 10 had no usable GPS time. Removing the 47 later flights left 43 sets with mean scaled rank 0.698 ($p = 0.0021$); also removing the 10 undated flights left 39 sets with 0.718 ($p = 0.0014$). The between-drone result therefore does not depend on post-crash flights, but the declared test did include them. The within-drone selection used the same date rule; the audit of those samples is reported in Section 5.6. The same-day pattern points to potential confounding by development or tuning sessions rather than progressive operational degradation: the "maintenance view" (vibration reflecting mechanical wear over time) competes directly with an "experimental airframe" view (poorly tuned or prototype drones vibrate more and crash during testing). Furthermore, matching was limited to flight controller hardware, PX4 major.minor version and vehicle type; it did not control for propellers, payload, airframe size, or operator identity (operator clustering by a location proxy is reported above). To test whether the 17 controls reused across the 20 strata artifi-

cially inflated significance, we re-evaluated the test via a two-stage cluster bootstrap resampling both case airframes ($n = 40$; the 60 sets come from 40 drones, 11 of which crashed more than once) and unique control airframes ($n = 94$) across 2,000 resamples: an earlier version of that bootstrap counted each resampled drone only once and gave [0.536, 0.789]; counted correctly, the interval is [0.50, 0.81] and the one-sided p is 0.012 (see above).

The vibration difference was carried by the older estimator metric ($\bar{r} = 0.68$, $p = 0.005$, 48 sets); within the original cohort, the exploratory subset with the newer accelerometer metric showed a smaller, non-significant difference ($\bar{r} = 0.62$, $p = 0.191$, 12 sets; Fig. 5). A declared replication protocol on newer accelerometer logs selected 26 sets, yielding 19 usable matched sets (9 sets pooled from the initial cohort and 10 new cases entirely outside the original test). On the 10 new cases, the mean scaled rank was $\bar{r} = 0.50$ (95% CI 0.20–0.75, $p = 0.5760$), which is inconclusive given the wide confidence interval. Pooling all 19 usable sets gives $\bar{r} = 0.58$ (95% CI 0.39–0.76, one-sided $p = 0.251$). A two-sample permutation test (100,000 draws) between the older estimator sets ($n = 48$, mean scaled rank 0.677) and newer accelerometer sets ($n = 19$, mean scaled rank 0.580) indicates that the difference in effect size is not statistically significant ($\Delta\bar{r} = 0.097$, two-sided permutation $p = 0.3221$), consistent with small-sample variability rather than structural divergence, though this heterogeneity test is underpowered given the sample sizes ($n = 19$ vs. 48). Statistical design power at $N = 19$ for an assumed effect of $\bar{r} = 0.67$ was 0.379 (and 0.118 if the true effect is $\bar{r} = 0.58$). Statistical sample sizing at one-sided $\alpha = 0.025$ indicates that detecting $\bar{r} = 0.67$ ($\Delta = 0.17$) with 80% power requires 55 matched sets; however, this assumes the discovery effect size was not inflated by winner's curse. If the true effect size is closer to the pooled estimate of $\bar{r} = 0.58$ ($\Delta = 0.08$), 247 sets are required. Because 247 matched sets exceed the capacity of the current single archive, resolving this effect definitively requires prospective multi-fleet partner deployments.

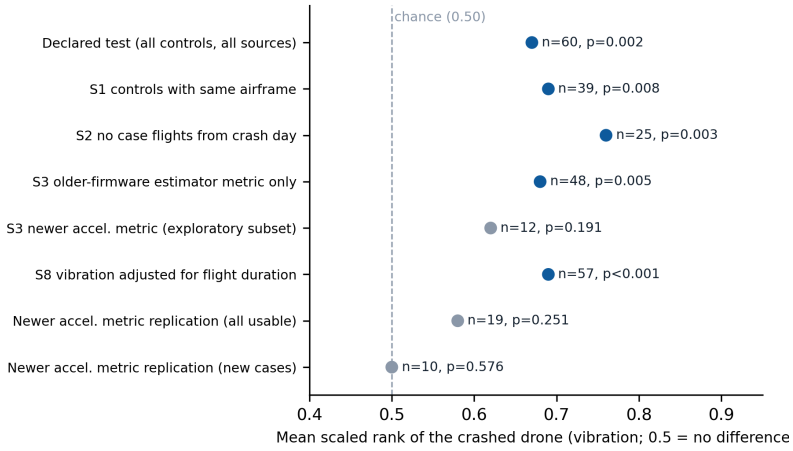


Figure 5: Between-drone vibration result under alternative specifications (exploratory). Specification labels: S1: airframe-matched controls; S2: excluding case flights from the crash day; S3: stratification by vibration measurement source (older-firmware estimator metric vs. newer accelerometer metric); S8: log-linear adjustment for flight duration. Blue: $p < 0.025$. These p-values treat sets as independent. †With one set per crashed airframe, median p is 0.026 (S1), 0.023 (S2), 0.019 (older estimator metric), 0.28 (newer accelerometer metric), 0.019 and 0.014 (GPS-ordered variants); S8 was not re-run and remains unclustered.

5.8. A published log checker under the same protocol (exploratory)

†To see whether these patterns are particular to our features and models, we re-evaluated an independent tool. FlightMD [41] is an open-source, rule-based PX4 log checker whose author validated it on 50 Flight Review logs; it is not peer-reviewed. Seven modules (oscillation, vibration, estimator, battery, GPS, parameters, motors) raise findings that the tool combines into a 0–100 health score. We ran its parser, analysers and score calculator unchanged (commit `306d0cb`), but not its full pipeline, which sends the log's GPS position to external weather and place-name services and has an optional language-model step that does not affect the score. The protocol, samples and wording rules were declared before any study log was scored (`fa5fb0b`); logs were cut at the crash moment as in Section 4.2. Because the tool is not trained, fold grouping does not apply; intervals come from a vehicle-cluster bootstrap.

On the constant cohort (89 crashes, 124 normal flights) the health score separated crashes from normal flights with AUC 0.79 [0.67, 0.87] on the whole

log, 0.74 [0.61, 0.83] up to the crash moment and 0.72 [0.58, 0.81] with the last 10 s removed, so it separates crash flights from other drones' flights before the crash window. Post-crash data and the last 10 s raised AUC by 0.048 [0.018, 0.089] and 0.025 [0.002, 0.054], both below the declared 0.05 rule (on the 24 tilt-defined multicopter crashes, 0.066 and 0.037). Most of the separation came from the estimator module (mean module score 63 in crashes against 92 in normal flights at $m = 10$ s). Against the drone's own earlier flights the same score gave no warning (crash-flight rank 0.48, $p = 0.66$, 46 drones). Taking "any critical finding" as an aircraft verdict, it reached a balanced accuracy of 0.52 [0.44, 0.61] against the uploader labels at the crash moment and 0.48 [0.38, 0.59] with the last 10 s removed, below the 0.70 pass mark; its score did not separate the labels either (AUC 0.47 [0.37, 0.57]). It rated 1.4% of normal flights below 60 ("Warning" or worse) but also only 9% of crashes. An independent tool therefore shows the same pattern as our own pipeline: separation between aircraft, no warning within an aircraft, and no attribution.

5.9. A known-cause control: injected faults on real flights (exploratory)

[†]The archive cannot separate a tool that fails from labels that fail. RflyMAD [11] can: its real flights carry faults injected in the PX4 firmware (motor efficiency loss and sensor faults on three multicopters, one operator), and each log records the injection in a dedicated topic, so the cause and its onset time are known. We downloaded the real-flight subset from its Kaggle copy and declared the analysis before scoring any log (`a9877c1`). Of 493 logs, 5 in fault folders had no fault code and were excluded, leaving 240 motor-fault, 197 sensor-fault and 51 fault-free flights (the Kaggle copy has 51 fault-free real flights, not the 84 the dataset page lists). Onset came from the log, not from the per-case workbooks, one of which described a different case. Fault-free flights were given a pseudo-onset at the median onset time of fault flights with the same flight pattern. Windows ran from take-off to the onset ("pre"), to 10 s after it, and to the end of the flight.

The declared attribution rule, unchanged, called 8 of 240 motor-fault flights (3%) and 10 of 197 sensor-fault flights (5%) aircraft-caused, against 2 of 51 fault-free flights (4%); its balanced accuracy for motor faults against fault-free flights was 0.50 [0.47, 0.52], below the 0.70 that would have shown it recognises known aircraft faults. Its signs are built for crashes (saturated motors, clipping, failsafes, warnings) and did not fire on a 15–20% loss of motor efficiency that the controller absorbed. FlightMD's health score did

not separate faulty from fault-free flights before the fault (AUC 0.47 [0.39, 0.55]) or with 10 s of it (0.51 [0.44, 0.58]), but did on the whole flight (0.69 [0.63, 0.75]).

The study's 11 features did the opposite. A logistic model separated faulty from fault-free flights at AUC 0.86 (permutation $p = 0.005$) using only data from *before* the fault was injected; including 10 s of the fault changed this by -0.008 [-0.033, 0.018], and the whole flight gave 0.82. The separation held within each flight pattern (pooled 0.88) and, in a check we had not declared, on the four days when both classes were flown (0.82, 163 flights). It came mainly from tilt and rate-tracking error (single-feature AUC 0.86 and 0.81): the fault flights were flown more aggressively before the injection. A fault-versus-normal classifier on this dataset can therefore score well with no fault present, which is the between-flight confounding of Section 5.5 in a setting where the cause is known. Both results bound the archive findings: the rule's failure there partly reflects signs that miss faults the controller absorbs, not only noisy labels; and a high crash-versus-normal AUC is weak evidence of a fault signal unless a pre-onset or same-flight baseline is reported.

5.10. Expert diagnoses (Q4)

Of 148 forum threads describing a real crash, the expert attributed it to the aircraft in 94 (64%), to the pilot in 9 (6%), to both in 2, and was unclear or gave no diagnosis in 43 (29%). Tuning or configuration was the cause named most often (26 of 94; 14 of the 47 confirmed by the poster), followed by GPS, compass or estimator problems (18) and motors, ESCs or propellers (16) (Fig. 6). Of 14 threads whose log carried an aircraft crash rating in the index, the expert agreed in 11 and was unclear in 3; none attributed the crash to the pilot. Download attempts across all 148 threads yielded 43 downloadable logs from 43 distinct threads (38 aircraft-fault and 5 pilot-fault; the remaining 105 had unhosted or expired external links). Of the 38 aircraft logs, exactly 35 were associated with checkable fault categories in Table 12 (12 tuning, 5 vibration, 5 GPS, 9 motor, 4 battery).

Table 12 compares the fault checks with the full flight window ($m = 0$ s) and with the last 10 s before the crash removed ($m = 10$ s). With the window ending 10 s before the crash, no check met the declared criterion. The tuning check illustrates the diagnostic leakage described in Section 5.3: across all candidate logs with the full window, the check fired on 10 of 12 (83.3%). Across the 9 constant tuning logs with duration ≥ 10 s surviving the 10 s cut-off without sample attrition, firing dropped from 7 of 9 (77.8%) with the

full window to 1 of 9 (11.1%) at $m = 10\,\mathrm{s}$ (6 logs lost firing, 0 gained; exact two-sided McNemar $p = 0.03125$). However, normal-flight firing also fell from 18/142 (13%) to 7/130 (5%), indicating that window shortening itself reduces cumulative exposure to threshold triggers alongside any crash-impact leakage. The tuning check itself is defined as rate-tracking RMS above $0.30\,\mathrm{rad/s}$, a quantity that loss of control raises by definition, so its drop shows that this one rule-based check depends on the crash window rather than measuring a general leakage effect. [†]Its drop was also not clearly larger than the normal-flight drop: among flights that fired at the full window and could be scored at both cut-offs, crash logs lost firing in 6 of 7 and normal flights in 4 of 11 (one-sided Fisher exact $p = 0.057$). Furthermore, this exploratory finding is descriptive rather than confirmatory: across the five diagnostic checks evaluated, an unadjusted $p = 0.031$ does not survive Bonferroni multiplicity correction ($p \approx 0.16$), and the other four checks exhibited low baseline sensitivity even on full windows ($\leq 25\%$, with 0/4 for motor and 0/2 for battery on paired logs), while tuning exceeded the declared 10% normal-flight false-alarm ceiling (13%). Thus, while the rate-tracking check depended on the crash window, claims that leakage is strictly confined to tuning must be tempered by the limited baseline sensitivity of the remaining checks.

To test diagnostic attribution on independently evaluated incidents, we evaluated the rule across all 43 downloadable logs behind expert forum threads (38 aircraft-fault and 5 pilot-fault). As noted in Section 3, cross-tabulating these 43 logs against archive ratings revealed that all 9 logs carrying an uploader rating were labeled `crash_sw_hw` (and all 9 were diagnosed as aircraft faults by experts, 100% agreement); the 5 expert pilot crashes were submitted unrated, which is consistent with, but not statistical evidence of, reluctance to self-identify pilot fault: all 5 pilot logs were unrated in the index against 28 of 38 aircraft logs unrated (one-sided Fisher exact $p = 0.25$). Evaluated on the 38 aircraft incidents, the fixed rule predicted "aircraft" in 18 (47.4%), "unknown" in 6 (15.8%), and erroneously blamed the "pilot" in 14 (36.8%). On decided aircraft cases ($n = 32$), the rule was wrong on 43.8% (14/32). Detailed inspection of flight mode sequences confirmed the failure mechanism: in all 14 misclassified logs, the vehicle was in a pilot-flown mode at the crash moment (`POSCTL` in 7 logs, `STAB` in 4, `ALTCTL` in 3; including log `e9034973`, which transitioned `POSCTL` $\rightarrow$ `AUTO_4` $\rightarrow$ `POSCTL` as the pilot attempted manual recovery). Because no hardware failsafe fired before impact, the rule defaulted to blaming the pilot.

Conversely, when evaluated on the 5 pilot-fault incidents, the fixed rule

predicted "aircraft" in 4 (80.0%) and "pilot" in only 1 (20.0%; pilot recall 1 of 5, exact Clopper–Pearson 95% CI 0.5–71.6%, Wilson 3.6–62.4%), misclassifying 4 of 5 pilot crashes because motor saturation, accelerometer clipping, or failsafe activation occurred as the vehicle struck the ground. Combining both classes across all 37 decided expert logs (32 aircraft, 5 pilot), the rule produced 18 incorrect decisions out of 37 (48.6% error rate), yielding a balanced accuracy of $(56.25\% + 20.0\%)/2 = 38.1\%$ (Bayesian 95% credible interval 0.25–0.61 from independent Jeffreys beta posteriors on the two recalls; exact 95% intervals for the recalls themselves are 37.7–73.6% for aircraft, 18 of 32, and 0.5–71.6% for pilot, 1 of 5). The interval contains chance (0.50) and lies below 0.70, the pass mark declared for the rule on uploader labels (not for this expert set), so the rule probably falls below that mark here but cannot be separated from chance. When evaluated on the subset of 34 logs with complete telemetry required by the statistical models (30 aircraft, 4 pilot), the rule decided 29 cases (25 aircraft, 4 pilot), predicting aircraft in 15 of 25 aircraft and 4 of 4 pilot, yielding a balanced accuracy of $(60.0\% + 0.0\%)/2 = 30.0\%$. Of the 30 complete aircraft logs, exactly 5 overlapped with the 148 modelled crashes in the training pool (all 5 rated `crash_sw_hw` in the index; 0 pilot logs overlapped). Evaluating the cross-validated logistic regression model across all 34 complete logs yielded predictions of "aircraft" in 27 of 30 aircraft logs (90.0%) and in all 4 pilot logs (0/4 pilot recall; covering 4 of the 5 pilot logs because one lacked actuator or setpoint topics), defaulting strongly toward its aircraft training prior (75.7% prevalence; 112/148) for a balanced accuracy of $(90.0\% + 0.0\%)/2 = 45.0\%$ and ROC AUC of 0.467 (95% bootstrap CI 0.211–0.733). When the 5 overlapping logs were excluded ($n = 29$: 25 aircraft, 4 pilot), the logistic regression model achieved AUC 0.450 (95% bootstrap CI 0.185–0.715); given the wide confidence interval spanning from 0.185 to 0.715 on 29 logs, this evaluation provides no evidence of discrimination and remains inconclusive. Across both heuristic rules and statistical models on this sample, log-based telemetry attribution showed no discriminative agreement above chance. We emphasize the caveat that this evaluation measures agreement between heuristic rules or models and expert community reading of the same telemetry, rather than ground truth: original posters explicitly confirmed the diagnosis in only 47 of 94 aircraft-fault threads (50.0%) and 5 of 9 pilot-fault threads (55.6%). Nonetheless, this independent evaluation illustrates the challenge of log-based attribution: neither the heuristic rule nor the statistical model demonstrated reliable agreement above chance.

Table 12: Fault checks on logs whose expert named the matching fault, with the full window ($m = 0$ s) and with the last 10 s before the crash removed ($m = 10$ s). Declared criterion: at least 50% of diagnosed logs and at most 10% of normal flights. Paired columns show constant logs surviving across both windows.

	Diagnosed ($m = 0$ s)		Diagnosed	Paired Δ	Normal comparison	
Expert-named fault	All logs	Paired	$m = 10$ s	(p-value)	$m = 0$ s	$m = 10$ s
Tuning or configuration	10/12 (83%)	7/9 (78%)	1/9 (11%)	-6 ($p = 0.031$)	18/142 (13%)	7/130 (5%)
Vibration or mounting	3/5 (60%)	3/5 (60%)	3/5 (60%)	0 ($p = 1.00$)	7/48 (15%)	4/38 (11%)
GPS, compass or estimator	1/5 (20%)	1/4 (25%)	1/4 (25%)	0 ($p = 1.00$)	10/143 (7%)	5/130 (4%)
Motor, ESC or propeller	1/9 (11%)	0/4	0/4	0 ($p = 1.00$)	8/143 (6%)	5/130 (4%)
Battery or power	0/4	0/2	0/2	0 ($p = 1.00$)	4/143 (3%)	1/130 (1%)

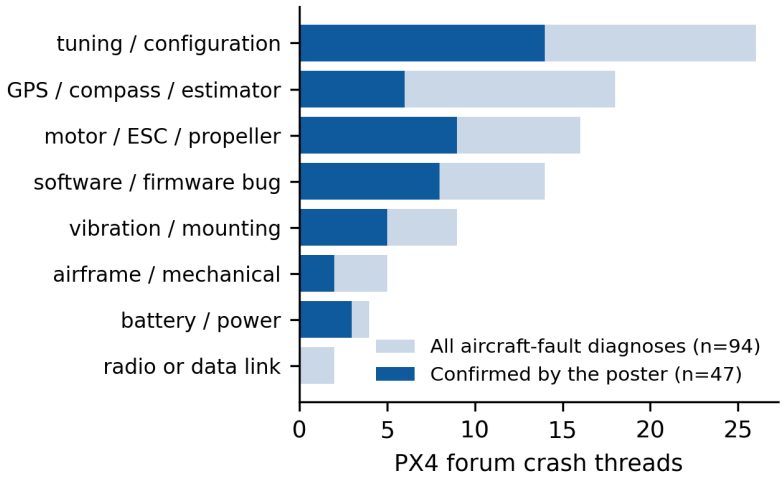


Figure 6: Causes named by experts in PX4 forum threads that attributed a crash to the aircraft.

5.11. Exploratory evaluation of language-model review panels (preliminary)

Because language-model assistants are increasingly proposed for automated flight log analysis, we conducted a preliminary exploratory evaluation on 60 author-generated fault cases using four local models (3–4B parameters) configured both as single agents and as multi-agent review panels (Table 14). While a fixed deterministic rule achieved 75% accuracy (naming the true cause in 35 of 50 active cases and correctly abstaining on 10 no-cause cases) with 0% error (referring 15 cases to human operators), local language models produced substantial error rates (10–63% wrong), suffered from mode collapse or complete abstention on active causes, and proved highly vulnerable to consensus manipulation: in a worst-case sensitivity sweep, a single compromised reviewer systematically flipped the panel's consensus decision in 45 to 56 of the 60 cases. In contrast, deterministic numeric telemetry extraction into code prevented direct prompt injection by construction across 40 hostile input strings. This exploratory comparison indicates that the tested small local models and review-panel configurations were unreliable on the 60 author-generated cases. Because the cases and fixed rule were developed by the same authors, the comparison is not an independent benchmark and should not be generalized to language models as a class or larger architectures. These preliminary results emphasize that multi-agent consensus voting without cryptographic proofs or fact-citation constraints cannot provide dependable safety assurance.

[†]*Blind real-log cases.* To remove the authors from the cases, we repeated the comparison on 97 real crash logs (declared `5b15715`, amended `7405264`): 35 from forum threads with expert-diagnosed causes and 62 from the independent relabelling (Section 5.2). No case was written by an author: the tool's own parser and fact builder extracted each case's facts from the last 20 s of the log, and the labels were mapped to the panel's causes before any run; 21 cases had a cause outside the panel's list (motor, vibration, software and similar), where the right answer is to defer. The four models were run on an NVIDIA RTX 4060 laptop with the same model versions (the runner's note lists the declared model IDs; the models were removed before `ollama list` was captured). The rule's answers and the labels matched a local recomputation on all 388 rows. Table 13 gives the results. The fixed rule, which looked flawless on the generated cases, named a wrong cause in 62 of 97 real crashes (64%) and was right in 26. By the declared criterion (more right answers and no more wrong ones than the rule) no panel beat it, so the conclusion that such panels are not recommended stands. Single agents

gemma3 and phi4-mini were never wrong, but only because they deferred about three quarters of the cases; with a single compromised reviewer the panel switched to the attacker's cause in 83 to 89 of 97 cases (16 for qwen2.5). On the 33 cases also run on the authors' Mac, the rule and the single agent gave identical answers on both machines, but the five-reviewer panel did so in only 16, so at temperature 0 the panel's verdict still depended on the hardware. On real logs, then, neither the rule nor any model named the cause reliably: the generated cases had flattered the rule.

Table 13: †Rule and language models on 97 real crash logs with independent labels (35 forum-expert, 62 blind relabelling; 21 with a cause outside the panel's list, where deferring is right): right (R), wrong (W) and deferred (Def) counts. Flips: cases in which one compromised reviewer moved the panel to its target cause.

Method	Single agent R / W / Def	Review panel R / W / Def	Flips / 97
Fixed rule (no model)	26 / 62 / 9	–	–
`gemma3:4b`	21 / 0 / 76	12 / 57 / 28	83
`phi4-mini`	23 / 0 / 74	17 / 40 / 40	89
`qwen2.5:3b`	27 / 70 / 0	26 / 69 / 2	16
`qwen3:4b`	23 / 28 / 46	20 / 35 / 42	88

Table 14: Language-model review panels on 60 generated cases: counts of right (R), wrong (W), and deferred (Def) cases, with coverage (Cov., share of cases decided) and accuracy when answering (Acc.). "Right" includes correctly leaving a no-cause case open. Single-agent gemma3 and phi4 answered only the 10 no-cause cases (0% coverage on active causes). Adversary flips show consensus changes caused by a single compromised reviewer in a worst-case sweep.

	Single agent			Review panel			Compromised
Method	R / W / Def	Cov.	Acc.	R / W / Def	Cov.	Acc.	Flips / 60
Fixed rule (no model)	45 / 0 / 15	75%	100%	–	–	–	–
`gemma3:4b`	10 / 0 / 50	17%[a]	100%[a]	3 / 36 / 21	65%	7.7%	49
`phi4-mini`	10 / 0 / 50	17%[a]	100%[a]	20 / 19 / 21	65%	51.3%	56
`qwen2.5:3b`	9 / 44 / 7	88%	17.0%	10 / 6 / 44	27%	62.5%	54
`qwen3:4b`	0 / 41 / 19	68%	0.0%	17 / 38 / 5	92%	30.9%	45

[a]Answered only on the 10 no-cause cases where abstention coincided with the ground truth; 0% coverage on active causes.

6. Design implications for fleet assurance

To translate these empirical findings into operational practice, we developed AeroFleet 2.0 (`https://github.com/AdityaPathare46/aerofleet`), an open-source, ground-side assurance tool for PX4 and ArduPilot fleets. The tool reads existing flight logs and configuration files without executing unverified autonomous decisions or contacting the aircraft in flight. Table 15 maps each empirical finding from Sections 3–5 directly to an assurance design implication.

In accordance with our negative attribution results (Q1–Q2), the tool presents a structured, chronological event review reporting evidence quality (dilution of precision, sensor health, and mode transitions) while strictly abstaining from issuing automated pilot-versus-aircraft liability verdicts. Responding to the vibration finding (Q3), the tool ranks airframes longitudinally against comparable peers within the same fleet to prioritize the top decile (10%) for physical maintenance inspection (measured against 412 non-crash drones: lift 1.77× [1.00, 2.60], about 76 inspections per crash, so offered as a weak aid, not a validated screen), rather than issuing per-flight crash alarms. Finally, to ensure evidentiary integrity over the lifecycle of each airframe, all reviews, approvals, and configuration checks are appended to a tamper-evident, Ed25519-signed record referencing SHA-256 digests of the original telemetry files.

6.1. Recommended reporting checklist for flight-log diagnostics

To support drone safety regulators, accident investigators, and data-driven diagnostic tool developers in avoiding the leakage and confounding traps quantified in this study, Table 16 outlines a nine-item recommended reporting checklist. Each item pairs a methodological recommendation with its empirical rationale and the specific failure mode observed in our experiments when that control is omitted.

Table 15: From empirical findings to design implications in fleet assurance.

Finding	Empirical result (Section)	Design implication for fleet assurance
Attribution failure	Across the telemetry features, rules, and models evaluated here, logs do not separate pilot from aircraft causes above chance (balanced accuracy 0.47 on held-out test, 0.381 on expert set, 95% CrI 0.25–0.61; §5.2, §5.10)	Present time-ordered event chronology and data quality metrics; abstain from automated fault verdicts.
Diagnostic leakage	Crash-window leakage was small for model AUC overall (0.014) but material among tilt-defined multicopter crashes (0.089 [0.019, 0.168]; §5.4), and present in one rule-based check (tuning sensitivity $7/9 \rightarrow 1/9$, exploratory; §5.3, §5.10)	Remove crash impact window ($m \geq 10\,\mathrm{s}$) and compare against aircraft operating baseline before diagnosing faults.
Between-drone vibration	Retrospective between-airframe association on older estimator metrics (mean rank 0.67; Holm $p = 0.016$ as declared, 0.08 when clustered by airframe; measured screening lift 1.77× [1.00, 2.60]; §5.6)	Show each drone's vibration against fleet peers as context for inspection, without a flag threshold presented as validated.
Comparison choice	Grouping folds by vehicle was the only material factor in the factorial audit; comparing a drone with its own earlier flights lowered AUC by a further +0.113 [0.029, 0.212] on the same 67 crashes (§5.5)	Evaluate airframe health against its own flight history, not cross-sectional fleet averages.
Audit integrity	Multi-agent consensus voting is vulnerable to reviewer manipulation in exploratory 60-case setup (§5.11)	Rely on deterministic numeric telemetry extraction and append reviews to a tamper-evident, signed log.

Table 16: Recommended checklist for leakage-controlled evaluation and reporting of UAV flight-log diagnostic models and accident analytics.

Checklist Item	Methodological Recommendation	Rationale and Empirical Failure Mode
1. Event-window removal	Remove ≥ 10 s prior to physical impact or ground tumble ($m \geq 10$ s)	Small for model AUC overall (Shapley share 0.014 [0.007, 0.023]) but material among tilt-defined multicopter crashes (0.089 [0.019, 0.168]; §5.4); present in one rule-based check (tuning 7/9 $\rightarrow$ 1/9; normal firing fell 4 of 11; Fisher $p = 0.057$). Confirm the crash moment before cutting; apply tilt rules to multicopters only.
2. Airframe-grouped folds	Partition cross-validation folds strictly by vehicle UUID	Prevents the model from recognising individual vehicles: grouping lowered AUC from 0.882 to 0.826 on the 287-flight sample and was the only material factor in the factorial audit (0.062 [0.034, 0.087]; §5.5).
3. Longitudinal baseline	Evaluate cases against the airframe's own pre-incident flights	Disentangles airframe confounding. Order flights by GPS time, not date: 85 of 210 crash-day "earlier" flights in this archive were flown after the crash; without them the within-drone model gave at most a weak signal (0.582 [0.525, 0.660]; §5.6).
4. Duration reporting	Report window durations of cases and comparisons; use duration matching only as a sensitivity analysis	Aborted or short crash flights create duration differences. A caliper lift of +0.110 [+0.028, +0.205] vanished once post-crash flights were removed (+0.086 [−0.064, +0.184]; §5.6).
5. Two-class expert ground truth	Validate against multi-coder independent diagnoses with reported agreement ($\kappa \geq 0.70$)	Uploader labels are noisy and biased; evaluation should quantify inter-rater reliability across both pilot-fault and aircraft-fault classes (§5.10).
6. Operational base-rate calibration	Measure flag rates on held-out non-case units, then report exact Bayes PPV, lift and inspections per event at fleet base rates	High ROC AUCs mask severe false-alarm fatigue under sparse base rates ($\leq 1\%$); assumed flag rates overstated lift here (3.3× assumed, 1.77× measured; §5.6).
7. Prospective protocol registration	Pre-specify hypotheses, sample funnels, features, models, seeds, and pass marks in version control	Eliminates p-hacking, outcome switching, and post-hoc rationalization of exploratory fluctuations (§4.1).
8. Persistent open artifact deposit	Deposit raw logs, extraction code, and derived feature tables in public archives	Ensures cryptographic provenance, auditability, and independent re-analysis across research groups.
9. Label audit[†]	Check that each label class actually contains the event it names (e.g. share of logs with a crash-like signature, per class)	61% of crash-rated multicopter logs show no crash-like tilt, and the share differs by responsibility label (23% vs 45%), so a classifier could separate labels without learning cause (§5.4).
10. Pre-onset baseline[†]	Where onset is known, report the same classifier on data before the onset	On 488 injected-fault flights our features reached AUC 0.86 before the fault and 0.85 with it, so the separation was not the fault (§5.9).

7. Discussion

Attribution. A declared rule, its held-out replication and cross-validated models all found no usable separation between pilot- and aircraft-caused crashes with these labels and features. Because pilot labels are sparse (only 36 pilot-labelled crashes in the modelling sample against 112 aircraft-labelled crashes), 80% power under the refitting permutation null requires an AUC of about 0.72; observed logistic and tree models gave AUC 0.42–0.59 with approximate upper bounds of at most 0.69. An operational attribution verdict would require an AUC of approximately 0.8 or higher, which these models do not approach. Furthermore, independent evaluation against 43 expert forum cases was statistically inconclusive (balanced accuracy 0.381, Bayesian 95% CrI 0.25–0.61; probably below the 0.70 pass mark declared for the uploader-label test, but not separable from chance; pilot recall 1 of 5, exact 95% CI 0.5–71.6%). Only the fixed rule was prospectively declared; the model results are exploratory, and with these sample sizes they bound, rather than rule out, a modest non-linear signal. Across the telemetry features, rules, models, and validation procedures evaluated here, we found no reliable above-chance separation of pilot-caused and aircraft-caused crashes. This result bounds the evidence available in the present dataset and setup; it does not establish that attribution is impossible under all sensors, labels, models, or operating conditions. While commercial and legal disputes often seek a binary verdict, aviation safety science demonstrates that accidents are multi-causal systemic events involving organizational, environmental and human factors [7]. Uploader labels are noisy, and the divergence between uploader pilot blame (32%) and expert forum diagnoses (6%) may reflect several untested mechanisms, including differing definitions of pilot responsibility among autopilot developers or differing thresholds for public fault attribution. A log records the combined physical outcome of pilot inputs, environmental disturbances and vehicle response; disentangling responsibility from telemetry alone is not supported by the data. Tools that issue automatic responsibility verdicts should be evaluated against independent investigations before adoption.

Leakage and confounding. While cross-sectional models on the discovery sample showed high apparent separation (AUC 0.88), grouping folds by vehicle lowered it to 0.826 and crash-window removal changed little (Table 8, regenerated). †That small window effect hides a subgroup: among multi-

copter crashes whose log shows a crash-like tilt, cutting the last 10 s mattered (0.089 [0.019, 0.168], 37 crashes; post hoc), and more so without tilt and stick features (0.131), so the effect is not just the defining tilt being removed. The overall figure is small partly because most crash logs show no clear crash (§5.4). †The same protocol applied to an independent, untrained tool (§5.8) gives a useful contrast: with nothing fitted, fold grouping cannot inflate it, yet its crash-vs-normal AUC (0.72 without the last 10 s) still came from differences between aircraft, mainly estimator health, and vanished against each drone's own earlier flights. The factorial audit (§5.5) shows why the picture depends on the sample: once comparison drones contribute several flights, grouping folds by vehicle becomes the only material source of optimism (0.062 [0.034, 0.087] of a 0.212 drop). Which crashes are scored and which normal flights they are compared with each carried a similar share, but neither could be isolated with 60–67 crashes. Separation against other aircraft persisted (0.67); only comparison with the same drone's own earlier flights brought it to chance. On the 372 flights of the multi-flight cohort, cross-sectional and within-drone evaluation gave overlapping intervals (0.546 and 0.559, regenerated), indicating that uncalipered matching yields no clear precursor signal in this cohort. A GPS-time audit then showed that 85 of 210 crash-day earlier flights had been flown after the crash. Without them the within-drone model gave 0.582 [0.525, 0.660] on stored features but 0.507 [0.436, 0.582] on recomputed features, so any within-drone signal is weak and pipeline-dependent. A duration-caliper lift reported in earlier versions (+0.110 [+0.028, +0.205]) is no longer clearly distinguishable from zero (+0.086 [−0.064, +0.184], 24 airframes); the interval still includes the earlier estimate, and with 24 airframes only a lift of about 0.18 could be detected, so the result is inconclusive rather than null; we withdraw the claim that duration differences mask precursors. Diagnostic checks were also *consistent with* crash-window leakage: specifically, tuning fault detection fell from 7/9 to 1/9 sensitivity (unadjusted $p = 0.031$ on paired logs) when the final 10 s were removed, accompanied by a drop in normal-flight firing from 13% to 5% which suggests that shorter windows reduce trigger exposure alongside leakage. Furthermore, this finding is exploratory, does not survive Bonferroni correction over five checks ($p \approx 0.16$), and the low baseline sensitivity of the remaining checks ($\leq 25\%$) leaves open whether other physical failure modes exhibit the same pattern. In interpreting feature signals, two operational definitions of motor saturation must be distinguished: the whole-flight cumulative summary feature used in predictive models (`sat_frac`, the fraction

of armed flight time with any motor at $\geq 95\%$ authority), versus the discrete threshold check used in the attribution rule (motor output at $\geq 95\%$ authority for $\geq 1.0\,\mathrm{s}$ cumulative during the final $10\,\mathrm{s}$ before t_c). Furthermore, while crash-flight analysis windows terminate at impact or tilt, normal-flight comparison windows end at disarm (encompassing the complete descent, flare, touchdown, and landing sequence), which contributes to duration differences and baseline feature exposure. Regardless of the precise mechanism, we recommend that diagnostic evaluations remove the crash window, report flight duration (whose effect we could not isolate), and compare against an aircraft's own operating baseline before claiming pre-crash anticipation.

Implications for work on public log archives[†]. PX4 Flight Review is the largest open source of real crash logs, and real PX4 logs underpin public anomaly datasets [12]. Our audit suggests four checks before using its crash ratings as labels. First, most crash-rated multicopter logs show no crash-like event, so a crash moment computed from tilt, and any analysis window built on it, is reliable only for the minority that do. Second, the share of logs with such an event differs by responsibility label, so a pilot-versus-aircraft classifier trained on these labels could appear to work by detecting tilt-type crashes, whether because recordings of pilot-caused crashes stop earlier or because those crashes involve less tilt. Third, a tilt rule is a crash criterion only for multicopters: fixed-wing and VTOL logs bank beyond 60° in normal flight, and older logs lack the flag that would exclude them, so vehicle type must come from the archive index. Fourth, dates cannot order flights within a day, so "earlier" flights must be ordered by GPS time. These are properties of the records rather than of a particular model.

Drone-level risk. The one positive finding concerns airframes, not flights, and is suggestive rather than established once the 60 sets are clustered by their 40 crashed airframes ($p = 0.012$, Holm-adjusted 0.08). Drones that later crashed vibrated more than comparable drones on ordinary flights, consistent with a maintenance view in which mechanical degradation precedes catastrophic failure [28]. However, two important qualifications apply. First, removing crash-day flights raised $\bar{r}$ from 0.67 to 0.76 ($p = 0.0027$, 25 sets), which cuts against a same-session flight disturbance explanation; however, because airframe tuning and hardware characteristics remain constant across days, this leaves open the alternative explanation that experimental, prototype, or chronically poorly-tuned airframes vibrate heavily across their entire operational lives and crash more frequently during testing. Second, the effect

was supported on the older estimator metric only; a declared replication on the newer accelerometer metric yielded an independent estimate of $\bar{r} = 0.50$ ($p = 0.5760$, 10 new sets) and pooled $\bar{r} = 0.58$ ($p = 0.251$, 19 sets), which is inconclusive and severely underpowered ($N = 247$ sets needed at true $\bar{r} = 0.58$ due to potential winner's curse). Third, the effect leans on a few operator sites (one set per site: median $p = 0.14$). Finally, against a reference of 412 non-crash drones with flag rates measured on held-out drones, a top-decile threshold flagged 13.7% of drones and gave a lift of only 1.77× [1.00, 2.60]; the earlier 3.3×, which assumed a 10% flag rate, is withdrawn. The observed between-airframe vibration difference is a retrospective screening signal, not evidence of a validated crash-warning capability. Its operational value requires prospective evaluation across multiple fleets, with sufficient control of flight context, airframe configuration, and maintenance history.

From findings to assurance design. Section 6 translates these empirical findings into design implications for fleet assurance: structured evidence review without automated verdicts, longitudinal airframe ranking, and tamper-evident telemetry records.

Language models. On this task small local models (3–4B parameters) added errors and voting vulnerabilities rather than accuracy, on the 60 author-generated cases and †on 97 blind real crash logs. The real cases also showed that the deterministic rule's perfect score on generated cases was a training-like score: on real logs it named a wrong cause in 64% of crashes. The comparison should not be generalized to language models as a class or larger architectures. While deterministic numeric parsing prevented direct prompt injection by construction, multi-agent voting consensus was easily manipulated by a single compromised reviewer in this setup.

Relation to prior datasets. Benchmarks built from injected faults [10, 11] answer whether a known fault can be detected; anomaly datasets from real logs [12] answer whether unusual segments can be found. Our results concern a different question, attribution of a real crash, and suggest that performance on the first two does not transfer to it: signals that mark a fault also mark the crash it causes and the pilot's reaction to it. Datasets that pair real crashes with independently verified causes would be needed to make progress on attribution, and our expert-diagnosis labels are a small first step. †Injected-fault data are still useful as a control: on RflyMAD's real flights our features separated faulty from fault-free flights before any fault existed

(AUC 0.86; Section 5.9), so evaluations on such datasets should report a pre-onset baseline alongside detection accuracy.

Records and regulation. Record-keeping rules ask operators to keep logs, maintenance records and occurrence reports [1, 2]. On our evidence, such records are most useful when they are kept per aircraft over time, so that an airframe's condition can be compared with its own history and with comparable airframes, rather than examined only after an accident.

Practical guidance. For operators and investigators we suggest: (i) treat automatic pilot-or-aircraft verdicts from logs with caution; (ii) use logs to assemble a time-ordered record of events for a human investigator; (iii) track vibration per drone across flights and inspect drones that are rough relative to comparable airframes; (iv) when evaluating any log-based diagnostic, remove the crash window and compare against the same drone's history.

Future work. Two studies have been declared in the repository before any data were collected: a prospective test in partner fleets, in which the monthly vibration ranking of AeroFleet 2.0 is written to the signed record before outcomes occur and crashes, unplanned repairs and defects found at inspection are compared between flagged and unflagged drones over six months; and a prospective evaluation with human operators. Stronger replication of the between-drone vibration effect on modern autopilots, evaluating operational safety impact across active fleets, testing sequence models or frequency-domain IMU features for transient precursors that whole-window summaries miss, and evaluating a larger set of crashes with causes verified by independent investigators would further strengthen the evidence.

8. Threats to validity

Archive self-selection. The public Flight Review archive reflects voluntary uploads from hobbyists, academic researchers, and commercial operators, introducing self-selection towards unusual flights, developer tuning sessions, and unexplained incidents. Crashed airframes and normal comparison flights differ substantially in flight duration (median 54 s vs. 388 s), firmware era, and operational envelope, which motivated our within-drone and matched-pair designs.

Labels. The uploader-assigned responsibility labels are not independently verified ground truth; they reflect the uploaders' own assessments, and normal flights are self-rated. †The independent relabelling (Section 5.2) was done by one reviewer from plots, not from the raw logs, and that reviewer's own reliability could not be measured; its near-zero agreement with the uploaders shows that the labels are contestable, not which party is right. With only 36 pilot-labelled crashes in the modelling sample (against 112 aircraft-labelled crashes), statistical power to detect subtle attribution signals is constrained. Across 148 expert forum crash threads, download attempts yielded 43 downloadable logs (38 aircraft-fault, 5 pilot-fault), enabling a two-class empirical validation that revealed an overall error rate of 48.6% on decided cases (balanced accuracy 0.381, Bayesian 95% CrI 0.25–0.61; pilot recall 1 of 5, exact Clopper–Pearson 95% CI 0.5–71.6%; uploader `crash_pilot` rating 0 of 43). However, forum diagnoses represent informed community opinion rather than official accident investigation reports; original posters confirmed the diagnosis in only 47 of 94 aircraft threads (50.0%) and 5 of 9 pilot threads (55.6%). Controls without a crash rating may have crashed unnoticed, biasing the between-drone test conservatively. Forum diagnoses achieved good inter-rater agreement ($kappa \geq 0.73$), but were restricted to public discussions.

Confounding and design. Matching on vehicle type, controller hardware and PX4 version leaves open differences in propellers, payload, flight style or operator identity, leaving potential unmodeled clustering. Results may not transfer to commercial ArduPilot fleets or to proprietary autopilots, even though the tool is designed to support both ArduPilot and PX4 logs.

Measurement. The crash moment is estimated; exploratory sensitivity checks across alternative pitch and roll tilt thresholds confirmed that the within-drone vibration result did not depend on the specific cutoff: 56 drones at 45° ($\bar{r} = 0.46, p = 0.79$), 61 drones at 60° ($\bar{r} = 0.48, p = 0.67$), and 64 drones at 75° ($\bar{r} = 0.50, p = 0.54$). The between-drone effect was shown mainly with the older vibration estimate and was not confirmed on the newer metric in a declared replication with 19 usable sets; this suggestive result needs a replication on newer firmware with one crash per airframe.

Protocols and registration. Protocols were declared and committed to version control in advance of analysis (confirmatory tests committed 20–24 October 2026: `63bc72e`, `af6fab2`, `4475c8c`, `0c8ee47`, `f911bcf` and `d5d56c7`,

8b417d2, d7a6574 on 20 and 23 October; 2148644 and 21af373 on 24 October; times in Table 4), and archived snapshots were deposited in Software Heritage (swh:1:rev:bad06ff1256865c7e63d44389aac174e0281291d, swh:1:rev:bbdd0420de4547ac7d67b84165292a46237b604d, swh:1:rev:624dd694e0d207dd9c44b1c9550faaf04453bc56, and swh:1:rev:3a9aa89417fa9ea2a2334e3a4794e2358fb713ee); however, because the earliest Software Heritage capture dates from 25 October 2026, commit timestamps prior to 25 October in self-hosted git remain author-controlled. The audits added after 25 October (from 3c34530 onwards, 26–28 October 2026) have no independent archive yet and will be archived with the Zenodo release; distinguishing declared protocols from formal third-party timestamped registries (such as the Open Science Framework). One amendment (lifting the within-drone firmware filter) was data-dependent after observing sample starvation ($n = 12$ under the initial ≥ 1.10 filter).

Features. Summary features over the whole window (percentiles, shares of time, RMS) were used; sequence models or frequency-domain IMU features might capture short precursors that summaries miss. This was not tested.

What a crash rating means†*.* Archive-wide, 61% of crash-rated multicopter logs show no crash-like tilt, and three reviewers disagreed on how many of those contain the crash at all (Fleiss' $\kappa = 0.33$; between 50% and 86% of crash-rated multicopter logs depending on the reviewer; Section 5.4). An uploader's crash rating does not show that the aircraft was lost, but the archive cannot show that it survived either. Of 490 crash-rated logs from real hardware with a vehicle identifier, 264 (54%) have a later-dated log from the same flight-controller serial (median 4.5 days later; 83 within one day); among the 40 crashed airframes of the between-drone test the share is 78% (median one day). Because a flight controller can be moved to a new frame, this shows that the controller was flown again, not that the aircraft survived; the same-day flights after the crash found by GPS time are consistent with minor incidents but do not prove them. The crash moment is also uncertain: in only 72 of the 148 modelled crash logs does tilt exceed 60°, and 31 of those are fixed-wing or VTOL logs where the rule is not a crash criterion; for the other 76, t_c falls back to the end of the armed period. Of all 148, 116 have t_c followed within 10 s by disarming or a landed or ground-contact flag, and 32 do not (clustering and refit audit). Two reviewers, one blind, agreed that the tilt-defined crash moment is mostly right (21 and 24 of 26; all 15

multicopter ones for both) and the fallback moment mostly wrong (2 and 6 of 24; $\kappa = 0.70$), but disagreed on whether fallback logs contain the crash at all ($\kappa = 0.32$; Section 5.4).

Flight order. Earlier flights were selected by calendar date. †The index date matched the GPS UTC date for 365 of 456 flights with a GPS time (80%; 90% within one day), so it is mostly the flight date, and none of 221 flights dated before the crash day started after the crash by GPS time; the problem is confined to same-day flights, which a date cannot order. GPS times showed that 47 of 95 crash-day exposure flights in the between-drone sample, and 85 of 210 crash-day earlier flights in the within-drone sample, were flown after the crash. The between-drone result and the within-drone rank tests held without them; the caliper lift did not, and the within-drone model result depends on the feature pipeline (§5.6). Flights on earlier dates are ordered by the index date only, and 18 crash-day flights had no usable GPS time.

Attrition. Across 148 real crash threads, download attempts yielded 43 downloadable logs; exactly 35 of the 73 threads with checkable expert aircraft diagnoses had downloadable logs (12 tuning, 5 vibration, 5 GPS, 9 motor, 4 battery), so several diagnostic comparisons in Table 12 have very small sample sizes ($n = 2$ to 9); its results are indicative rather than definitive.

Power and sizing. Some comparisons are small (14 pilot-caused matched sets, 12 sets with the newer vibration metric, five vibration-diagnosed logs). Null results there are not evidence of no effect. For the newer accelerometer vibration metric, the independent sample of 10 new cases yielded $\bar{r} = 0.50$ ($p = 0.5760$), and the pooled 19 sets yielded $\bar{r} = 0.58$ ($p = 0.251$). Statistical design power at $N = 19$ for an assumed effect of $\bar{r} = 0.67$ was 0.379 (and 0.118 if $\bar{r} = 0.58$). Detecting $\bar{r} = 0.67$ with 80% power requires 55 sets assuming the discovery effect was uninflated, but if winner's curse inflated the initial estimate and the true effect is $\bar{r} = 0.58$, 247 matched sets are required. Because 247 sets exceed the capacity of the public archive, definitive replication requires multi-fleet partner deployments.

Language models. The AI comparison is preliminary: the experiments use 60 author-generated cases and relatively small local models (3–4B parameters). The fixed rule's performance may be optimistic on cases created by the same authors who designed the rule (0 wrong answers is close to a training score). †The blind real-log set confirms this: the rule was wrong in 64% of 97 real

cases. Its labels come from forum experts and one reviewer working from plots, the facts cover only the last 20 s of each log, and the panel's verdict changed between two machines in half the cases checked. Larger models or alternative prompt architectures were not tested.

Injected faults[†]*.* The RflyMAD control uses three airframes flown by one team, faults injected in firmware (mostly a 15–20% motor efficiency loss or sensor noise) that rarely end in a crash, and stratified rather than vehicle-grouped folds, because three airframes are too few to group by. Its pre-fault separation shows that fault and fault-free flights were flown differently; it does not show how large that effect is in other datasets. The rule's low sensitivity there concerns faults the controller absorbs; it does not show the rule would miss a fault that ends in a crash.

The tool. AeroFleet 2.0 is an open-source evidence-review tool whose operational safety benefit has not yet been demonstrated in field deployment. The software verifies parameter baselines and longitudinal vibration ranks deterministically from telemetry files; prospective evaluation of the vibration ranking in operational fleets and evaluation with human operators remain future work (Sections 6 and 7).

9. Conclusion

Across 590 crash-rated logs from a 479,547-flight PX4 archive, flight logs did not support pilot-versus-aircraft verdicts. A fixed rule reached 0.47 balanced accuracy on held-out crashes, and exploratory models stayed near chance (AUC 0.42–0.59; approximate upper bounds at most 0.69), short of the roughly 0.8 an operational verdict needs; with 36 pilot-labelled crashes, only an AUC of about 0.72 could be detected with 80% power. Agreement with 43 expert-diagnosed logs was inconclusive (balanced accuracy 0.38, Bayesian 95% CrI 0.25–0.61). A blind reviewer who is not an author agreed with the uploader ratings no better than chance, and the rule did no better against that reviewer's labels, so the failure is not explained by noisy labels; the labels themselves are contestable.

Part of what looks like crash anticipation comes from how an evaluation is built. In a factorial audit of 32 settings, the apparent AUC of 0.88 fell by 0.21 across settings, of which 0.06 came from grouping folds by vehicle, the only factor that could be isolated; and crash-window leakage was small overall but material among the minority of crashes whose log shows a crash-like

tilt. The archive's labels are also weaker than assumed: most crash-rated multicopter logs show no crash-like tilt, where the computed crash moment is usually wrong, and pilot-rated logs show one less often than aircraft-rated ones. Ordering flights by archive date failed for same-day flights: by GPS time, 44–55% of dated crash-day "earlier" flights were flown after the crash. Against each drone's own earlier flights, the crash flight showed no warning in vibration or rate tracking. A published open-source log checker, run unchanged under the same protocol, showed the same three results: separation from other drones' flights, no warning within a drone, and attribution at chance. On real flights with injected faults of known onset, our features separated faulty from fault-free flights before the fault was injected, and the declared rule recognised almost none of the known motor faults.

What the logs did show is a suggestive between-airframe signal: drones that later crashed vibrated more than matched drones on ordinary flights on older estimator metrics (mean rank 0.67; $p = 0.012$ clustered by airframe, Holm-adjusted 0.08), and replication on newer metrics was inconclusive. Its value for choosing which drones to inspect is not established: with flag rates measured on 412 non-crash drones, a top-decile vibration ranking gave a lift of 1.77× [1.00, 2.60], and the effect weakens when each operator site counts once.

Log analytics for drone fleets should therefore present evidence rather than issue verdicts; ranking airframes by vibration is a hypothesis for prospective testing, and should be evaluated with vehicle-grouped folds, comparisons against each aircraft's own history, and flights ordered by GPS time. AeroFleet 2.0 implements this approach as an open-source tool.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper. The authors develop the open-source AeroFleet 2.0 tool described in Section 6; it is not commercial.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used an AI assistant (Claude, Anthropic) to write analysis code, draft and revise text, and assist

the recording of forum diagnoses. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

Data availability

Selection scripts, analysis code, the declared hypotheses with their commit history, derived features, expert labels (without forum text), and the source code of the AeroFleet 2.0 tool are available at `https://github.com/AdityaPathare46/aerofleet`; a DOI-archived release will be provided on acceptance. Raw logs are public on PX4 Flight Review and are referenced by identifier.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Appendix A. Exploratory analyses added after the declared protocol

To preserve the boundary between pre-declared confirmatory hypotheses (committed to version control on 20–24 October 2026; Table 4) and exploratory analyses added after the declared protocol [36], Table A.17 documents each such analysis, its motivation, key results, git commit hash, first archive date in Software Heritage, and analysis run date. They are marked with † throughout the text and tables. None of these exploratory additions altered the conclusions of the pre-declared tests. They bounded between-airframe confounding, tested other model types, quantified fleet screening, and audited the crash records themselves: the crash moment, the uploader labels against log content, and the order of flights.

Table A.17: Exploratory analyses added after the declared protocol, their motivation, key results, git commit hashes, the four commits archived in Software Heritage on 25 October 2026 (`swh:1:rev:bad06ff1256865c7e63d44389aac174e0281291d`, `swh:1:rev:bbdd0420de4547ac7d67b84165292a46237b604d`, `swh:1:rev:624dd694e0d207dd9c44b1c9550faaf04453bc56`, and `swh:1:rev:3a9aa89417fa9ea2a2334e3a4794e2358fb713ee`), and analysis run dates. Later commits are not yet archived.

Analysis	Motivation	Key result	Commit	Run
1,000-permutation null (Task B)†	Calibrate detection threshold	Null mean 0.497; observed AUC upper bounds 0.510–0.619 (80% power MDE 0.75); superseded by the refit permutation null (upper bounds $\leq$ 0.69, MDE 0.72)	`bad06ff`	2026-10-25
Depth-1 decision stumps (Task B)†	Test single-feature non-linear splits	Stumps AUC 0.430 $\rightarrow$ 0.413 at $m = 10$ s (upper bounds 0.534 and 0.528); superseded by the refit null (0.447 and 0.481)	`bad06ff`	2026-10-25
Constant-subset leak curve†	Exclude sample attrition ($n = 213$)	AUC declines smoothly 0.895 $\rightarrow$ 0.868 ($m =$ 10 s)	`bbdd042`	2026-10-25
Paired diagnostic leakage test†	Test leakage on constant logs ($n = 9$)	Tuning check 7/9 $\rightarrow$ 1/9, exact McNemar $p = 0.03125$	`bbdd042`	2026-10-25
Same-sample decomposition†	Account for airframe & duration confounds	Changing comparison from other aircraft to same aircraft lowers AUC 0.83–0.88 to 0.53–0.57; superseded by the factorial audit, in which fold grouping was the only material factor	`eb463b5`	2026-10-25
Poisson-binomial set null†	Account for mixed set sizes ($n = 60$)	Exact null rate 40.28%; observed 55.0% ($p =$ 0.0137)	`bad06ff`	2026-10-25
Screening PPV and CIs†	Quantify operational fleet value	Exploratory illustration: fleet PPV 2.26–2.43% (exact Bayes) at assumed 10% flag rate (3.3$\times$ lift); withdrawn, superseded by the measured reference below	`bad06ff`	2026-10-25
Sensitivity by vibration source†	Compare older vs. newer metrics	Older: 32/48 (66.7%); newer: 6/12 (50.0%); pooled newer $\bar{r} = 0.58$ ($p = 0.25$)	`624dd69`	2026-10-25
Independent 43-log validation†	Evaluate against independent causes	Balanced accuracy 0.381 (95% CrI 0.25–0.61); pilot recall 1/5; inconclusive	`3a9aa89`	2026-10-25
Cohort selection step 4b†	Isolate comparison vs. cohort effect	67 multi-flight crashes vs. other-aircraft controls: AUC 0.522 [0.413, 0.615]; superseded by the factorial audit (mixed extractors)	`f152d07`	2026-10-26
Within-drone duration caliper†	Test duration confounding in cohort	Caliper $\leq$ 0.25 matching lifts within-drone AUC +0.110 (0.532 to 0.642)	`f152d07`	2026-10-26
Depth-3 tree bounds across cuts†	Bound non-linear model performance	Upper bounds 0.519–0.724; superseded by the refit permutation null (0.54–0.69)	`dfbabaf`	2026-10-26
Fixed-score permutation null†	Test CV fold-splitting variance	Fixed score SD 0.055 matches asymptotic theory (0.056); 0.090 reflects refitting variance	`dfbabaf`	2026-10-26
Control cluster adjustment†	Test control reuse robustness	Mean rank 0.665, 95% cluster CI [0.587, 0.744]; heterogeneity $p = 0.322$	`dfbabaf`	2026-10-26
Single vs. multi-history crashes†	Test maiden-flight artifact hypothesis	Single-history ($n = 45$) AUC 0.835 vs. multi-history ($n = 103$) 0.891 at $m = 0$ s; cross-sectional separation is archive-wide	`aa1d7c6`	2026-10-26

Analysis	Motivation	Key result	Commit	Run
Duration caliper gradient & CI†	Quantify caliper sensitivity	Lift curve $\epsilon \in [0.10, 1.00]$; paired cluster bootstrap 95% CI for $\Delta = +0.125$ is [+0.074, +0.238]; superseded by the reproducible re-run [+0.028, +0.205]	`aa1d7c6`	2026-10-26
Two-stage cluster bootstrap & GPS†	Audit same-day timing & clustering	Two-stage cluster CI [0.536, 0.789] (resampled drones counted once; corrected to [0.50, 0.81]); same-day check compared dates only; superseded by the GPS flight-order audit	`aa1d7c6`	2026-10-26
Rule balanced accuracy across cuts†	Audit attribution rule across margins	Feature-based proxy of the rule: BA 0.46 [0.38, 0.54] ($m = 0$ s) to 0.45, 0.39, 0.37, 0.35 [0.25, 0.45] ($m = 30$); below 0.5 from low pilot recall, not undecided cases	`aa1d7c6`	2026-10-26
Archive-wide label audit†	All 520 crash-rated logs	39% of 341 readable multicopter logs show a tilt event; share containing the crash 50–86% depending on reviewer; pilot-rated 23% vs aircraft-rated 45% ($p = 0.0003$)	`09cf99c`, `18b0eff`	2026-10-27
Label, detector and tilt-defined audit†	Era-adjusted label link; objective crash detector; window analyses on tilt-defined crashes	Adjusted OR 2.08 [1.17, 3.69]; detector failed gate (κ 0.72 and 0.38; 64% false alarms); tilt-defined window effect 0.067 [0.015, 0.131]	`35103da`	2026-10-28
Aircraft-type and label-link audit†	Round-19 review: tilt set mixed aircraft types; mechanical-effect concern; tilt as label proxy; multi-cue detector	Multicopter tilt-defined effect 0.089 [0.019, 0.168], 0.131 [0.042, 0.226] without tilt/stick; all 15 reviewed multicopter tilt moments correct for both reviewers; tilt-alone AUC 0.53; within-stratum attribution 0.63 ($p = 0.10$) and 0.62 ($p = 0.11$); rule proxy BA 0.46 [0.38, 0.54]; multi-cue detector failed gate (κ 0.82 and 0.32; 32% false alarms)	`9650a69`	2026-10-29
FlightMD re-evaluation†	Test a published open-source log checker under this protocol	Crash vs normal AUC 0.79 whole log, 0.74 at t_c, 0.72 [0.58, 0.81] at $m = 10$ s; window drops below 0.05; attribution BA 0.52 [0.44, 0.61]; within-drone rank 0.48 ($p = 0.66$); 1.4% of normal flights below 60	`fa5fb0b`	2026-10-29
RflyMAD injected faults†	Known-cause positive control on 488 real flights	Rule BA 0.50 [0.47, 0.52] (3% of motor faults called aircraft); FlightMD 0.47 pre-fault, 0.69 whole flight; features AUC 0.86 pre-fault ($p = 0.005$), fault adds −0.008; within flight pattern 0.88	`a9877c1`	2026-10-29
Third rater†	Non-author rater on the blind crash-moment sheet	Moment right: Fleiss' κ 0.71 [0.53, 0.86]; contains crash: 0.33 [0.05, 0.58]; sided with blind reviewer on 11 of 13 disputed logs; majority archive estimate 80% [68, 92]	`fdb3b7c`	2026-11-01
Independent relabelling†	Blind non-author relabelling of 72 crashes from plots	Agreement with uploader $\kappa = -0.10$ [−0.34, 0.17]; 16/36 pilot-rated called aircraft; rule vs. reviewer BA 0.55 [0.42, 0.66]; FlightMD 0.52; tilt AUC 0.54	`fa5fb0b`	2026-11-01
Operator clustering†	Cluster the between-drone test by operator (location proxy, 2 km)	40 crashed airframes from 23 sites; site-clustered $p = 0.006$ (Holm 0.042); one set per site median $p = 0.14$; kept as suggestive (deviation from declared rule)	`cfb79b9`	2026-11-01

Analysis	Motivation	Key result	Commit	Run
Screening reference†	Rebuild reference from 412 non-crash drones; measure flag rates	Nominal 10% flagged 13.7%; lift 1.77× [1.00, 2.60]; AUC 0.65; not shown to beat random inspection	`4961234`	2026-11-01
Blind real-log panel test†	Rule and four local models on 97 real crash logs with independent labels	Rule 26 right / 62 wrong; no panel beats it; single agents defer ∼77%; flips 16–89; Mac vs RTX panel agreement 16/33	`5b15715`, `7405264`	2026-11-02
Crash-moment review†	Check t_c by eye (two authors, second blind)	t_c right: agreement 85%, $\kappa = 0.70$; tilt-defined correct 21 and 24 of 26 (multicopters 15 of 15 for both), fallback 2 and 6 of 24; whether fallback logs contain the crash $\kappa = 0.32$	`97974ad`, `59c8212`	2026-10-28
Crash-moment and date audit†	Confirmed crashes, dates vs GPS, grouped leak curve, clustered variants, stumps	40 of 148 crashes confirmed; leak drop 0.035 overall, 0.103 confirmed; index date = GPS date for 80%, no cross-day misorder; clustered variant p 0.014–0.026 (newer metric 0.28); stumps 0.447/0.481	`48c2fa7`	2026-10-27
Clustering and refit audit†	Refit nulls, clustered between-drone p, paired comparison, fixed-seed Table 8, re-flights, crash moment	Between-drone clustered $p = 0.012$ (Holm 0.08); other minus own flights +0.113 [0.029, 0.212]; Step 1 grouping 0.882 → 0.826; 54% of crash-rated logs have a later log from the same flight controller; Task B refit-null upper bounds ≤ 0.69	`6fff27d`, `016dafe`	2026-10-27
Post-crash flight audit†	GPS-order the within-drone samples	85 of 210 crash-day earlier flights followed the crash; without them rank tests unchanged, within-drone model 0.582 [0.525, 0.660], caliper lift +0.086 [−0.064, +0.184] (masking withdrawn), same-aircraft 0.507	`0d38f5d`	2026-10-27
Flight-order audit†	Caliper without duration-linked features; GPS ordering	Lift +0.077 [+0.013, +0.204] without three features; 47 of 95 crash-day exposure flights followed the crash; between-drone 0.698 ($p = 0.0021$, 43 sets) without them	`0e26db0`, `187f570`	2026-10-27
Factorial optimism audit†	Isolate what the AUC depends on	2×2: no cause isolated; drop 0.212 [0.062, 0.335], only grouped folds material (0.062)	`3c34530`, `f6ba424`	2026-10-27

References

[1] European Commission, Commission Implementing Regulation (EU) 2019/947 of 24 May 2019 on the rules and procedures for the operation of unmanned aircraft, Official Journal of the European Union, L 152, 11 June 2019 (2019).

[2] European Union Aviation Safety Agency, Easy Access Rules for Unmanned Aircraft Systems (Regulations (EU) 2019/947 and 2019/945), EASA, Cologne, revision of June 2026 (2026).

[3] Ministry of Civil Aviation, Government of India, The Drone Rules, 2021, G.S.R. 589(E), Gazette of India, 25 August 2021 (2021).

[4] Federal Aviation Administration, 14 CFR Part 107: Small Unmanned Aircraft Systems, Electronic Code of Federal Regulations, as of 21 October 2026 (2026).

[5] V. S. A. Duvvuru, B. Zhang, M. Vierhauser, A. Agrawal, LLM-agents driven automated simulation testing and analysis of small uncrewed aerial systems, in: 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE), 2025, pp. 385–397. doi:10.1109/ICSE55347.2025.00223.

[6] S. Silalahi, T. Ahmad, H. Studiawan, F. Breitinger, DroPTC: Sentence-level drone flight log forensics using contrastive learning and explainable AI, Forensic Science International: Digital Investigation 56 (2026) 302051. doi:10.1016/j.fsidi.2026.302051.

[7] D. A. Wiegmann, S. A. Shappell, A Human Error Approach to Aviation Accident Analysis: The Human Factors Analysis and Classification System, Ashgate, Aldershot, 2003.

[8] S. Kaufman, S. Rosset, C. Perlich, O. Stitelman, Leakage in data mining: Formulation, detection, and avoidance, ACM Transactions on Knowledge Discovery from Data 6 (4) (2012) 15:1–15:21. doi:10.1145/2382577.2382579.

[9] S. Kapoor, A. Narayanan, Leakage and the reproducibility crisis in machine-learning-based science, Patterns 4 (9) (2023) 100804. doi:10.1016/j.patter.2023.100804.

[10] A. Keipour, M. Mousaei, S. Scherer, ALFA: A dataset for UAV fault and anomaly detection, The International Journal of Robotics Research 40 (2–3) (2021) 515–520. doi:10.1177/0278364920966642.

[11] X. Le, B. Jin, G. Cui, X. Dai, Q. Quan, RflyMAD: A dataset for multicopter fault detection and health assessment, The International Journal of Robotics Research 44 (7) (2025) 1081–1092. doi:10.1177/02783649241305153.

[12] A. Kabaoglu, S. Sariel, UAV-SEAD: State estimation anomaly dataset for UAVs, arXiv preprint https://arxiv.org/abs/2602.13900 (2026).

[13] G. Wild, J. Murray, G. Baxter, Exploring civil drone accidents and incidents to help prevent potential air disasters, Aerospace 3 (3) (2016) 22. doi:10.3390/aerospace3030022.

[14] Air Accidents Investigation Branch, Annual Safety Review 2025, Tech. rep., AAIB, Department for Transport, Farnborough, UK, published 4 June 2026 (2026).

[15] L. Meier, D. Honegger, M. Pollefeys, PX4: A node-based multithreaded open source robotics framework for deeply embedded platforms, in: 2015 IEEE International Conference on Robotics and Automation (ICRA), 2015, pp. 6235–6240. doi:10.1109/ICRA.2015.7140074.

[16] PX4 Development Team, Flight Review log archive and pyulog ULog parser, https://review.px4.io; pyulog v1.2.4, https://github.com/PX4/pyulog; BSD-3-Clause (2026).

[17] J. Wei, H. Li, Y. Li, Y. Zhao, D. Yao, B. Lu, X. Ye, S.-K. Im, Y. Wang, X. Yang, AeroTSBoost: Temporal-statistical boosting for real-world UAV telemetry anomaly mining, arXiv preprint https://arxiv.org/abs/2605.25639 (2026).

[18] K. Kose, J. Wing, N. A. Kose, C. Guadarrama-Trejo, A. Sowers, A. Rasheed, A UAV testbed for diagnosing hardware vulnerabilities: Quantifying sim-to-real discrepancies in PX4 flight logs, Sensors 26 (10) (2026) 3188. doi:10.3390/s26103188.
URL https://www.mdpi.com/1424-8220/26/10/3188

[19] D. Wang, S. Li, G. Xiao, Y. Liu, Y. Sui, P. He, M. R. Lyu, An exploratory investigation of log anomalies in unmanned aerial vehicles, in: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE), 2024, pp. 2593–2605. doi:10.1145/3597503.3639186.

[20] R. Puchalski, W. Giernacki, UAV fault detection methods, state-of-the-art, Drones 6 (11) (2022) 330. doi:10.3390/drones6110330.

[21] E. Khalastchi, M. Kalech, On fault detection and diagnosis in robotic systems, ACM Computing Surveys 51 (1) (2018) 9:1–9:24. doi:10.1145/3146389.

[22] E. Zibaei, R. Borth, Building causal models for finding actual causes of unmanned aerial vehicle failures, Frontiers in Robotics and AI 11 (2024) 1123762. doi:10.3389/frobt.2024.1123762.

[23] D. Wang, S. Li, G. Xiao, Y. Liu, Y. Sui, An exploratory study of autopilot software bugs in unmanned aerial vehicles, in: Proceedings of the 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE), 2021, pp. 20–31. doi:10.1145/3468264.3468559.

[24] T. Kim, C. H. Kim, A. Ozen, F. Fei, Z. Tu, X. Zhang, X. Deng, D. J. Tian, D. Xu, From control model to program: Investigating robotic aerial vehicle accidents with MAYDAY, in: Proceedings of the 29th USENIX Security Symposium, USENIX Association, 2020.
URL https://www.usenix.org/conference/usenixsecurity20/presentation/kim

[25] M. Seong, W. Yun, C. Lee, S. Lee, J. Park, Flight log analysis and recovery for open-source drones: Focusing on ArduPilot, PX4, and Betaflight, Computers and Electrical Engineering 135 (2026) 111144. doi:10.1016/j.compeleceng.2026.111144.
URL https://linkinghub.elsevier.com/retrieve/pii/S0045790626002168

[26] R. Hakani, A. Rawat, A. Kumar, Enhancing UAV safety through battery health monitoring and flight log-based crash diagnostics, IEEE Sensors Reviews (2025). doi:10.1109/SR.2025.3597582.

[27] E. Mantas, C. Patsakis, GRYPHON: Drone forensics in dataflash and telemetry logs, in: Advances in Information and Computer Security, Lecture Notes in Computer Science, Springer, 2019, pp. 377–390. doi:10.1007/978-3-030-26834-3_22.

[28] E. Petritoli, F. Leccese, L. Ciani, Reliability and maintenance analysis of unmanned aerial vehicles, Sensors 18 (9) (2018) 3171. doi:10.3390/s18093171.

[29] A. P. Tvaryanas, W. T. Thompson, S. H. Constable, Human factors in remotely piloted aircraft operations: HFACS analysis of 221 mishaps over 10 years, Aviation, Space, and Environmental Medicine 77 (7) (2006) 724–732.

[30] H. Li, J. Ma, Z. Wang, L. Chen, R. Xu, J. Ai, UFA: An LLM-driven UAV forensic intelligent agent for low-altitude public security, Expert Systems with Applications 327 (2026) 132859. doi:10.1016/j.eswa.2026.132859.
URL https://linkinghub.elsevier.com/retrieve/pii/S0957417426017720

[31] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, I. Mordatch, Improving factuality and reasoning in language models through multiagent debate, in: Proceedings of the 41st International Conference on Machine Learning (ICML), Vol. 235 of PMLR, 2024, pp. 11733–11763.
URL https://proceedings.mlr.press/v235/du24e.html

[32] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, M. Fritz, Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection, in: Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec), 2023, pp. 79–90. doi:10.1145/3605764.3623985.

[33] J. P. Vieira, V. A. Bauler, R. K. Rosa, D. Silva, Towards a more realistic evaluation of machine learning models for bearing fault diagnosis, arXiv:2509.22267 (2025). doi:10.48550/arXiv.2509.22267.

[34] B. Frénay, M. Verleysen, Classification in the presence of label noise: A survey, IEEE Transactions on Neural Networks and Learning Systems 25 (5) (2014) 845–869. doi:10.1109/TNNLS.2013.2292894.

[35] E. A. Stuart, Matching methods for causal inference: A review and a look forward, Statistical Science 25 (1) (2010) 1–21. doi:10.1214/09-STS313.

[36] B. A. Nosek, C. R. Ebersole, A. C. DeHaven, D. T. Mellor, The preregistration revolution, Proceedings of the National Academy of Sciences 115 (11) (2018) 2600–2606. doi:10.1073/pnas.1708274114.

[37] J. A. Hanley, B. J. McNeil, The meaning and use of the area under a receiver operating characteristic (ROC) curve, Radiology 143 (1) (1982) 29–36. doi:10.1148/radiology.143.1.7063747.

[38] H. B. Mann, D. R. Whitney, On a test of whether one of two random variables is stochastically larger than the other, The Annals of Mathematical Statistics 18 (1) (1947) 50–60. doi:10.1214/aoms/1177730491.

[39] B. Efron, R. J. Tibshirani, An Introduction to the Bootstrap, Chapman & Hall, New York, 1993.

[40] S. Holm, A simple sequentially rejective multiple test procedure, Scandinavian Journal of Statistics 6 (2) (1979) 65–70.
URL https://www.jstor.org/stable/4615733

[41] P. Behl, FlightMD: open-source diagnostic tool for PX4, ArduPilot and MAVLink flight logs, GitHub repository, commit 306d0cb, mIT licence; accessed 29 October 2026 (2026).
URL https://github.com/Praddyx15/FlightMD